МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Казанский национальный исследовательский
технический университет им. А. Н. Туполева — КАИ

Институт компьютерных технологий и защиты информации
Кафедра прикладной математики и информатики

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине «Компьютерное зрение»

Прикладные методы компьютерного зрения
Анализ изображений Treponema pallidum

Отчёт о выполненных экспериментах

Выполнил: студент группы 4299
Андреев А. К.

Проверила: к. т. н., доцент
Бикмуллина И. И.

Казань, 2026

Содержание

Цель работы....3
1. Общая схема и подготовка....4
2. Ключевые точки: YOLO-pose и MediaPipe....10
3. Трекинг, дисторсия, оптический поток, GAN и OCR....17
4. Поиск похожих изображений....33
5. Обучение метрики сходства....39
6. Нейросети без обучения: zero-shot....44
7. Поиск момента в видео RuTube и интерфейс Streamlit....48
8. Автоматическая подготовка разметки для CVAT....54
9. Учёт экспериментов в MLflow....58
10. Полный цикл YOLO-pose....64
11. Ответы на контрольные вопросы....69
Заключение....72
Список использованных источников....74

Цель работы

Освоить прикладные методы компьютерного зрения: оценку ключевых точек, сопровождение объектов, коррекцию дисторсии, оптический поток, сверхразрешение, распознавание текста, поиск по признакам, обучение метрики, zero-shot-модели, поиск момента в видео, подготовку разметки и учёт экспериментов. Основной объект исследования — изображения бледной трепонемы Treponema pallidum. Структура практических разделов соответствует частям 2–10 методических указаний к лабораторной работе № 4.

Для каждого метода рассматриваются входные данные, последовательность вычислений, программа и выходные артефакты. Численные результаты сопоставляются с изображениями: высокая скорость, наличие рамки или большое сходство эмбеддингов сами по себе не означают точного определения границ и вида бактерии.

1. Общая схема и подготовка

1.1. Объект исследования и особенности изображений

Бледная трепонема относится к спирохетам: её характерный визуальный признак — тонкая вытянутая спиралевидная форма. При анализе изображений внимание уделяется извилистому контуру, непрерывности осевой линии и изменению формы между кадрами. На двумерной проекции отдельные участки могут перекрываться или выходить за границу изображения. Поэтому связная область маски далее называется видимым фрагментом, а число таких областей характеризует изображение, а не число биологических клеток.

Основное изображение взято по заданной ссылке pcr.news. Это цветная иллюстрация размером 870 × 540 пикселей с хорошо различимыми спиралевидными объектами. Она используется для контролируемых геометрических опытов, выделения маски, формирования ключевых точек и обучения pose-моделей. Дополнительные микрофотографии Treponema pallidum позволяют сравнить внешний вид объекта при разных способах получения изображения. Видео t07.ogv содержит микроскопическую съёмку спирохет из исторического фильма Жана Командона.

Работа с бактерией в этой лабораторной означает обработку цифровых изображений. Сначала сохраняются исходные файлы и сведения об источниках, затем выбираются представление цвета и пространственная область анализа. Для цветной иллюстрации удобна маска по каналам BGR и HSV; для микроскопического видео — яркость, градиенты и временное смещение. Все длины и скорости рассчитываются в пикселях и пикселях в секунду. Физический масштаб задаётся только при наличии известной калибровки конкретного изображения.

Перед выделением объектов изображение читается OpenCV в порядке BGR. Для нейросетей и вывода через Matplotlib оно преобразуется в RGB. Маска очищается от небольших областей и отверстий, затем выполняется

морфологическое замыкание. У каждой крупной компоненты определяются рамка, площадь и осевая линия. Для спиралевидного объекта центральная точка выбирается по половине длины этой линии: она сохраняет связь с формой объекта при изгибе.

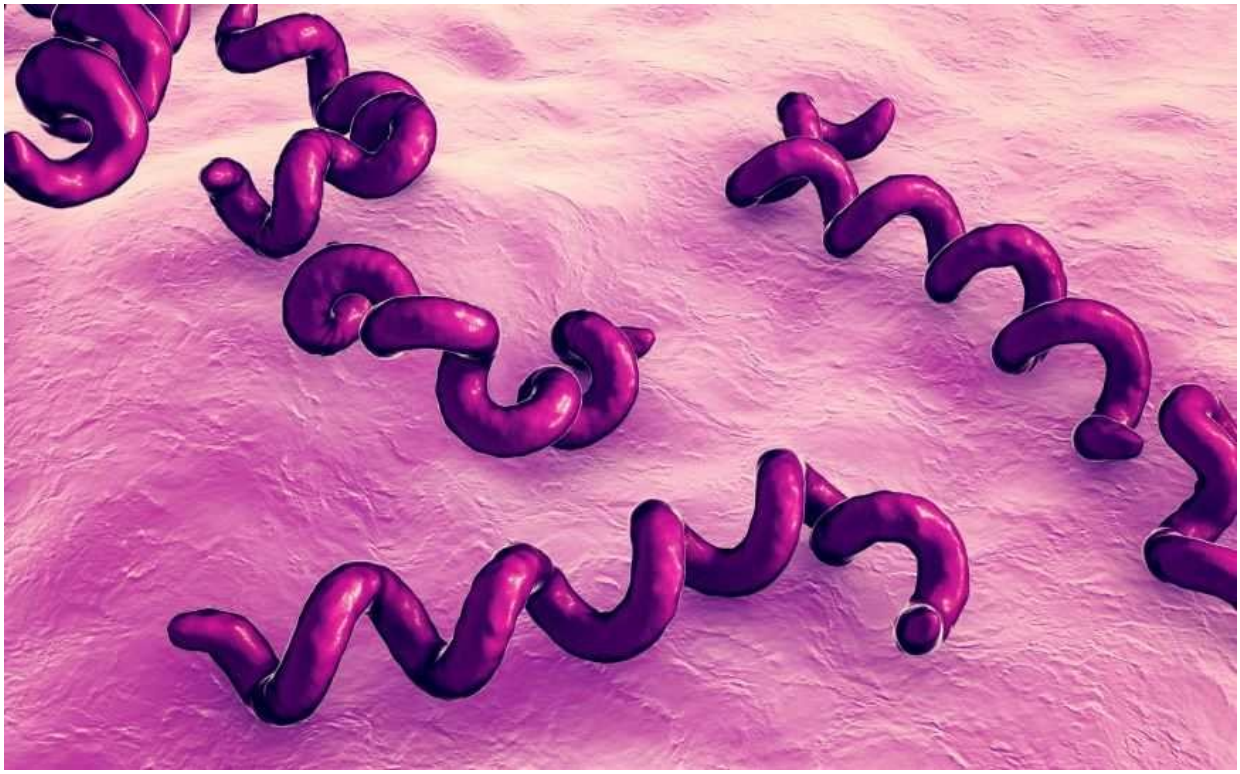


Рисунок 1 — Основное изображение Treponema pallidum, использованное в работе

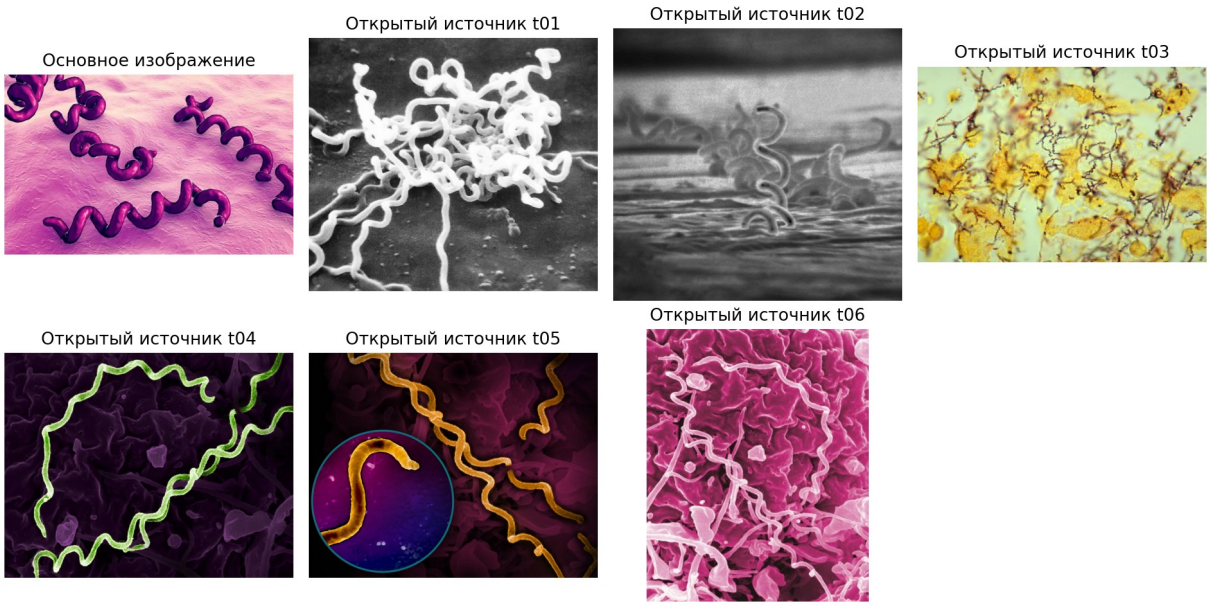


Рисунок 2 — Дополнительные изображения Treponema pallidum из открытых источников

Таблица 1

Файл	**Размер**	**Материал и источник**
t01.jpg	572 × 500	Photo Credit: Content Providers(s): CDC/ Dr. David Cox; Public domain
t02.jpg	531 × 500	Photo credit: Content providers(s): CDC / Dr. David Cox; Public domain
t03.png	734 × 497	CDC; Public domain
t04.jpg	2816 × 2190	NIAID; Public domain
t05.jpg	4123 × 3206	NIAID; CC BY 2.0
t06.jpg	391 × 480	NIAID; CC BY 2.0
t07.ogv	360 × 288	Jean Comandon; Public domain

1.2. Данные, разбиение и программная среда

Дополнительные палочковидные бактерии и кокки используются как классы сравнения для поиска и классификации. Контрольный снимок людей bus.jpg нужен для проверки моделей, обученных на человеческом скелете. Эти материалы позволяют отделить работоспособность программного конвейера от переноса модели на новый тип объекта.

Таблица 2

Набор	**Состав**	**Назначение**
Основной снимок	870 × 540, 6 крупных компонент	Маска, скелет, геометрия, zero-shot
Галерея поиска	80 изображений 224 × 224	40 train, 15 validation, 25 test
Набор YOLO-pose	32 train, 4 validation, 4 test	Варианты шести видимых фрагментов
Видео t07.ogv	Кадры 360 × 288; отрезок с 36 с	Трекинг и оптический поток
Видео RuTube	81 индексированный кадр	Поиск по тексту в Streamlit

В галерее разделение выполняется по исходным файлам: фрагменты одного источника попадают в одну часть набора. Близкие варианты t04 и t05 оставлены вместе в train. В pose-наборе сначала распределяются шесть исходных компонент, а затем выполняется преобразование масштаба и положения. Четыре компоненты формируют train, одна validation и одна test. Такой набор проверяет учебный цикл на одной иллюстрации; его метрики описывают именно эти видимые фрагменты.

Вычисления выполнены на CPU с фиксированным seed = 7. Используются PyTorch, Ultralytics, Transformers, OpenCV contrib, MediaPipe Tasks, scikit-image, scikit-learn, MLflow, EasyOCR и Tesseract. В окружении оставляется один пакет OpenCV с contrib-модулями, поскольку SOT-трекеры доступны через cv2.legacy. Вспомогательные пути и параметры потоков вынесены в общий модуль experiment_common.py.

Алгоритм выполнения. 1) Сохранить исходные изображения и видео. 2) Проверить размеры и порядок каналов. 3) Сформировать маску и крупные компоненты. 4) Разделить источники на train, validation и test. 5) Задать seed и каталоги результатов. 6) Сохранять численные наблюдения в JSON, изображения в PNG, модели в PT и историю обучения в CSV.

Входные файлы находятся в data/raw, подготовленные наборы — в data/final_gallery и data/final_pose. Выходные данные отдельных опытов сохранены в results/final. Каталоги с загружаемыми весами и служебным кешем отделены от результатов. Для воспроизведения сначала устанавливаются зависимости из requirements.txt, затем подготавливаются официальные веса моделей. Программы ниже приведены полностью; общий модуль используется всеми экспериментальными сценариями.

Листинг — Общие пути, сохранение результатов и выделение точек осевой линии. Полный исходный текст файла experiment_common.py.

```
"""Shared paths and reproducible CPU configuration for the laboratory experiments."""
import os
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]
LEGACY = ROOT.parent / 'lab4'
CACHE = ROOT / '.cache'
OUT = ROOT / 'results/final'
OUT.mkdir(parents=True, exist_ok=True)
for name, value in {'HF_HOME': CACHE/'hf', 'TORCH_HOME': CACHE/'torch',
                    'YOLO_CONFIG_DIR': CACHE/'yolo', 'MPLCONFIGDIR': CACHE/'mpl',
                    'XDG_CACHE_HOME': CACHE}.items():
    os.environ[name] = str(value)
os.environ['HF_HUB_DISABLE_XET'] = '1'
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['OMP_NUM_THREADS'] = '2'
import json
import numpy as np
import torch
import cv2
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
torch.set_num_threads(2)
cv2.setNumThreads(2)
```

```
torch.manual_seed(7)
np.random.seed(7)

def save(name, value):
    (OUT / name).write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding='utf-8')

def panel(name, images, titles, cols=3):
    rows = (len(images)+cols-1)//cols
    fig, axes = plt.subplots(rows, cols, figsize=(4*cols, 3.4*rows), squeeze=False)
    for i, axis in enumerate(axes.flat):
        if i < len(images):
            im = images[i]
            axis.imshow(im, cmap='gray' if im.ndim == 2 else None)
            axis.set_title(titles[i], fontsize=10)
        axis.axis('off')
    fig.tight_layout()
    fig.savefig(OUT / name, dpi=160)
    plt.close(fig)

def rgb(path):
    from PIL import Image
    return np.array(Image.open(path).convert('RGB'))

def skeleton_points(mask):
    """Return start/midpoint/end along the longest reachable skeleton path."""
    from skimage.morphology import skeletonize
    from scipy.sparse import coo_matrix
    from scipy.sparse.csgraph import dijkstra
    yy,xx=np.nonzero(skeletonize(mask>0))
    coordinates=np.column_stack([xx,yy])
    lookup={(int(x),int(y)):i for i,(x,y) in enumerate(coordinates)}
    rows=[];columns=[];weights=[]
    for (x,y),i in lookup.items():
        for dx,dy in [(1,0),(-1,0),(0,1),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]:
            j=lookup.get((x+dx,y+dy))
            if j is not
None:rows.append(i);columns.append(j);weights.append(float(np.hypot(dx,dy)))
    graph=coo_matrix((weights,(rows,columns)),shape=(len(xx),len(xx))).tocsr()
    distance=dijkstra(graph,indices=0)
    start=int(np.argmax(np.where(np.isfinite(distance),distance,-1)))
    distance,previous=dijkstra(graph,indices=start,return_predecessors=True)
    end=int(np.argmax(np.where(np.isfinite(distance),distance,-1)))
    indices=[end]
    while indices[-1]!=start:
        indices.append(int(previous[indices[-1]]))
    path=coordinates[indices[::-1]].astype(float)
    if tuple(path[0])>tuple(path[-1]):path=path[::-1]
    length=np.r_[0,np.cumsum(np.linalg.norm(np.diff(path,axis=0),axis=1))]
    midpoint=path[int(np.searchsorted(length,length[-1]/2))]
    return np.stack([path[0],midpoint,path[-1]]),path
```

Листинг — Подготовка цветовой маски основного изображения. Полный исходный текст файла prepare_primary.py.

```
"""Prepare a colour mask from the local Treponema illustration."""
from experiment_common import *
from skimage.morphology import remove_small_holes, remove_small_objects, skeletonize

def main():
    image = cv2.imread(str(ROOT/'data/raw/treponema_primary.jpg'))
    if image is None:
        raise FileNotFoundError('data/raw/treponema_primary.jpg')
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    mask = (image[:, :, 1] < 55) & (hsv[:, :, 1] > 100)
    mask = remove_small_objects(mask, max_size=119)
    mask = remove_small_holes(mask, max_size=149)
    mask = cv2.morphologyEx(mask.astype('uint8'), cv2.MORPH_CLOSE,
                            np.ones((3, 3), np.uint8))
    skeleton = skeletonize(mask > 0)
    _, _, stats, _ = cv2.connectedComponentsWithStats(mask)
    cv2.imwrite(str(ROOT/'results/primary_mask.png'), mask * 255)
    cv2.imwrite(str(ROOT/'results/primary_skeleton.png'), skeleton.astype('uint8') * 255)
    save('primary_preparation.json', {
        'shape': list(image.shape[:2]),
```

```
        'mask_fraction': float(mask.mean()),
        'components_over_450': int((stats[1:, 4] > 450).sum())})

if __name__ == '__main__':
    main()
```

Листинг — Подготовка весов моделей. Полный исходный текст файла prepare_models.py.

```
"""Download official model artifacts; never access RuTube."""
from experiment_common import CACHE
import argparse
from huggingface_hub import hf_hub_download
import requests

def main():
    p = argparse.ArgumentParser()
    p.add_argument('group', choices=['clip','vision'])
    args = p.parse_args()
    if args.group == 'clip':
        from transformers import CLIPModel, CLIPProcessor
        CLIPModel.from_pretrained('openai/clip-vit-base-patch32', use_safetensors=True)
        CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
    else:
        from ultralytics import YOLO
        for name in ['yolov8s-pose.pt', 'yolo11s-pose.pt', 'yolov8s-worldv2.pt', 'FastSAM-s.pt']:
            YOLO(str(CACHE/name))
        path = CACHE/'pose_landmarker_lite.task'
        if not path.exists():
            url =
'https://storage.googleapis.com/mediapipe-models/pose_landmarker/pose_landmarker_lite/float16/1/
pose_landmarker_lite.task'
            response = requests.get(url, timeout=120)
            response.raise_for_status()
            path.write_bytes(response.content)
        hf_hub_download('ai-forever/Real-ESRGAN', 'RealESRGAN_x4.pth')
    print('Prepared', args.group, flush=True)

if __name__ == '__main__':
    main()
```

2. Ключевые точки: YOLO-pose и MediaPipe

2.1. Контроль на изображении людей и перенос на бактерии

Ключевая точка задаёт место объекта с постоянным смыслом. Для человека стандарт COCO содержит 17 точек, а MediaPipe Pose — 33. Для каждого найденного объекта получаются координаты точек и оценки уверенности. Контроль на изображении людей показывает, какие структуры распознаёт исходная модель до адаптации к бактериям.

Алгоритм выполнения. 1) Загрузить YOLOv8s-pose. 2) Обработать bus.jpg при conf = 0,5. 3) Обработать основное изображение при conf = 0,5; 0,25; 0,05. 4) Сохранить число рамок и изображение со скелетами. 5) Создать MediaPipe PoseLandmarker с моделью lite. 6) Преобразовать координаты MediaPipe из долей ширины и высоты в пиксели и отобразить точки.

Таблица 3

Вход и модель	**Порог**	**Объектов**	**Точек в схеме**
YOLOv8s-pose, люди	0,5	4	17
YOLOv8s-pose, трепонема	0.5	0	17
YOLOv8s-pose, трепонема	0.25	0	17
YOLOv8s-pose, трепонема	0.05	5	17
MediaPipe lite, люди	По умолчанию	1	33
MediaPipe lite, трепонема	По умолчанию	0	33

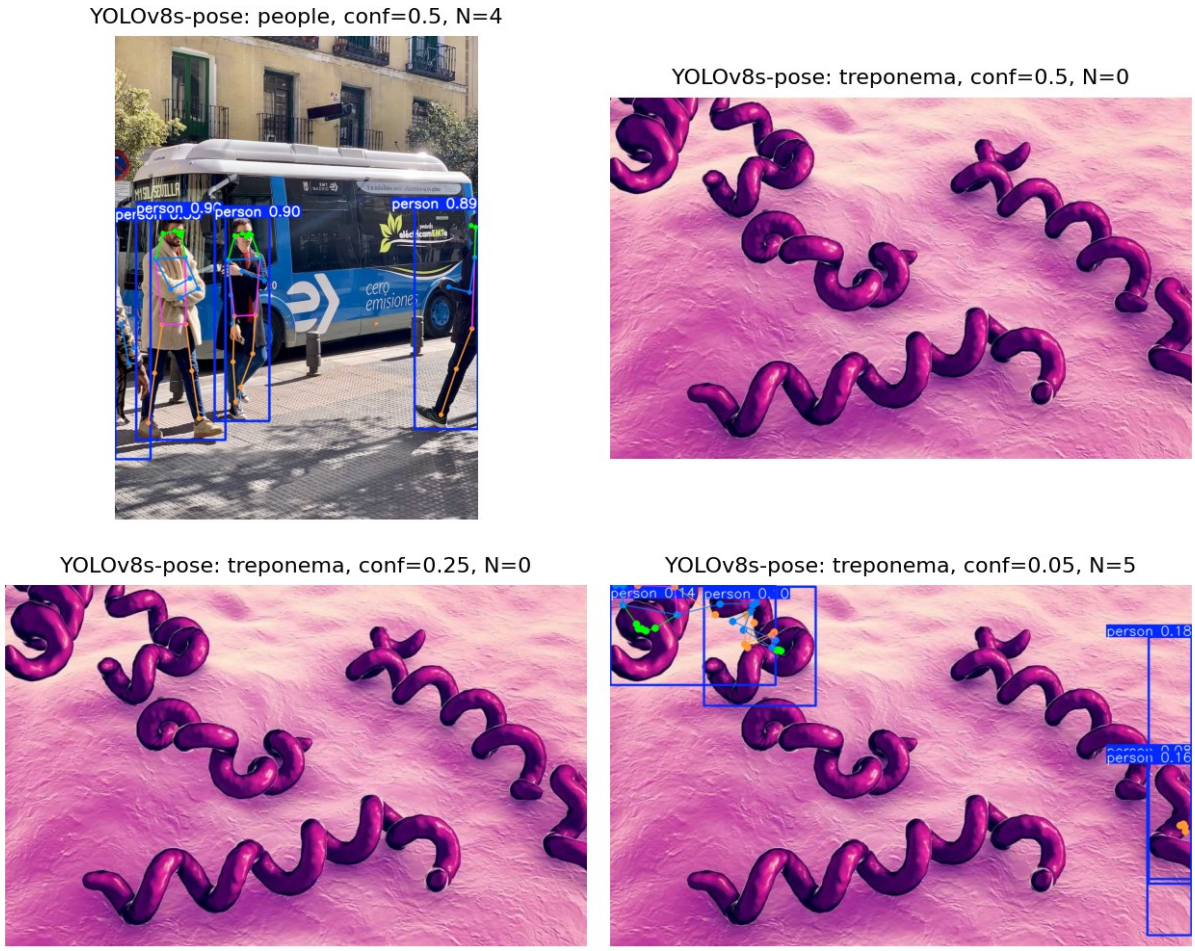


Рисунок 3 — YOLOv8s-pose: люди и Treponema pallidum при трёх порогах уверенности

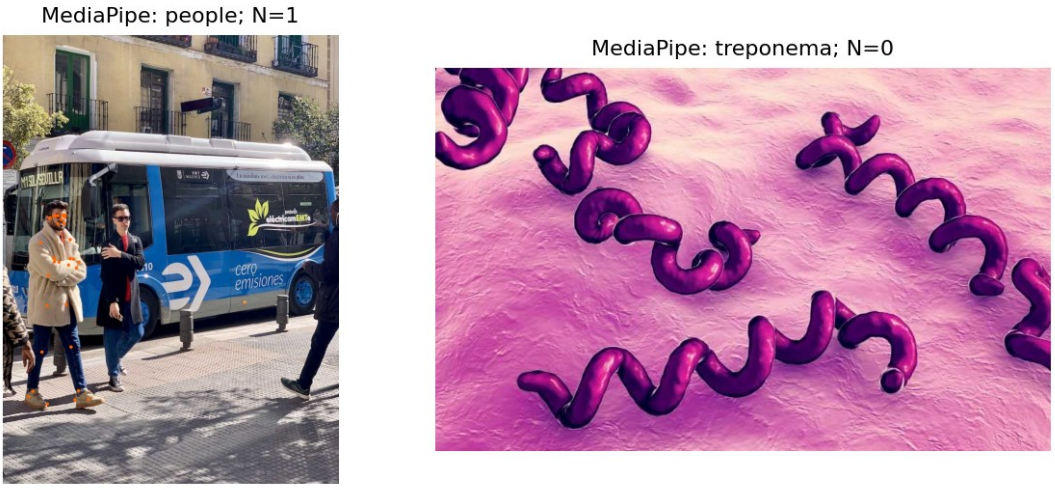


Рисунок 4 — MediaPipe Pose Landmarker: точки тела на контрольном снимке и обработка изображения трепонемы

На изображении людей YOLOv8s-pose выделяет четыре объекта, MediaPipe — одного человека. На иллюстрации трепонемы при порогах 0,5 и

0,25 число рамок YOLO равно нулю, а при 0,05 возникают пять срабатываний человеческой модели. Это демонстрирует влияние порога: снижение conf увеличивает число кандидатов, однако значения точек продолжают соответствовать человеческой схеме. Для бактерии нужна собственная разметка и адаптация pose-модели.

2.2. Схема «начало — середина — конец»

Для каждого видимого фрагмента выделяется тонкий скелет маски. Его пиксели образуют граф: соседние по стороне пиксели соединены ребром длины 1, по диагонали — √2. Два прохода алгоритма Дейкстры дают длинный достижимый путь по осевой линии. Точка 0 лежит на одном конце пути, точка 1 — на половине его накопленной длины, точка 2 — на другом конце. Порядок концов фиксируется сравнением координат.

Методичка предлагает главную ось полигона для палочковидной формы. Здесь та же схема из трёх точек адаптирована к спиралевидной трепонеме через осевую линию. Расстояние между крайними точками L характеризует хорду; сумма длин шагов скелета характеризует длину видимого извилистого фрагмента. Эти величины различаются и используются в разных расчётах.



Рисунок 5 — Маска и три ключевые точки, расположенные вдоль осевой линии трепонемы

Таблица 4

Компонента	**Площадь, px²**	**Длина осевой линии, px**	**Начало; середина; конец**
1	12587	249.62	(6, 98); (64, 80); (14, 0)
2	14012	329.29	(154, 117); (250, 89); (174, 4)

Компонента	Площадь, px²	Длина осевой линии, px	Начало; середина; конец
3	18867	510.62	(551, 146); (685, 173); (758, 321)
4	17005	375.88	(251, 213); (326, 281); (447, 238)
5	6681	173.27	(837, 288); (845, 362); (869, 425)
6	36779	822.02	(112, 456); (407, 446); (663, 447)

Строка YOLO-pose содержит 14 чисел: класс, центр и размеры рамки, затем три тройки (x, y, v). Координаты нормируются на размеры изображения. Видимость v = 2 означает видимую точку, v = 1 — закрытую, v = 0 — отсутствующую. В конфигурации kpt_shape = [3, 3], flip_idx = [2, 1, 0]. При отражении по горизонтали меняются местами концы, центральная точка остаётся центральной.

2.3. Сопровождение и счётчик изменения формы

Для последовательности кадров программа связывает рамки через ByteTrack и хранит состояние по идентификатору трека. Изогнутость вычисляется как $\kappa = |(p_2 - p_0) \times (p_1 - p_0)| / L^2$, где $L = \|p_2 - p_0\|$. Наивный счётчик увеличивается в каждом кадре с изогнутой формой. Счётчик переходов увеличивается только при переходе из прямого состояния в изогнутое. Порог входа 0,10 и порог выхода 0,07 образуют гистерезис, уменьшающий дрожание состояния.

Алгоритм выполнения. 1) Открыть локальный видеофайл. 2) Для каждого кадра вызвать model.track с persist=True. 3) Сопоставить точки и ID трека. 4) Вычислить изогнутость. 5) Обновить состояние и два счётчика. 6) Вычислить среднее значение FPS по окну последних 15 кадров. 7) Сохранить наблюдения и контрольные кадры.

Листинг run_video_checks.py содержит обработку локального видео, поиск по тексту и изображению и реализацию счётчиков. Его выходной формат включает ID, номер кадра, изогнутость, число переходов и FPS. Количественное сравнение выполненного сопровождения четырьмя SOT-трекерами представлено в разделе 3; оценки pose-моделей на отложенных изображениях — в разделе 10.

Листинг — Поиск в локальном видео и счётчики по ключевым точкам. Полный исходный текст файла run_video_checks.py.

```
"""Check local video search and pose tracking with a hysteresis bend counter."""
from experiment_common import *
from video_index import model, search
from ultralytics import YOLO
from PIL import Image
from collections import deque
import time

def main():
    directory=ROOT/'data/video_index'
    net,processor=model()
    query='Treponema pallidum spirochetes moving in a microscope field'
    text_hits=search(directory,query,net,processor)
    image_hits=search(directory,'',net,processor,image=Image.open(ROOT/'data/raw/
treponema_primary.jpg'))
    panel('video_search_actual.png',[rgb(directory/r['path']) for r in text_hits+image_hits],
          [f"{kind}: {r['seconds']:.0f} c, {r['cosine']:.3f}"
           for kind,hits in [('Text',text_hits),('Image',image_hits)] for r in hits])
    save('video_search_actual.json',{'query':query,'text':text_hits,'image':image_hits})
    net=YOLO(OUT/'pose_v8s_best.pt')
    cap=cv2.VideoCapture(str(ROOT/'data/raw/t07.ogv'));cap.set(cv2.CAP_PROP_POS_MSEC,36000)
    state={};naive=0;flagged=0;fps=deque(maxlen=15);observations=[];images=[];titles=[]
    for number in range(36):
        ok,frame=cap.read()
        if not ok:break
        begin=time.perf_counter()

result=net.track(frame,persist=True,tracker='bytetrack.yaml',conf=.1,imgsz=192,verbose=False)[0]
        if result.boxes.id is not None:
            for track_id,points in
zip(result.boxes.id.int().tolist(),result.keypoints.xy.cpu().numpy()):
                a,b,c=points;axis=c-a;length=np.linalg.norm(axis)
                bend=float(abs(axis[0]*(b-a)[1]-axis[1]*(b-a)[0])/max(length*length,1e-6))
                before=state.get(track_id,False);now=bend>(.07 if before else .10)
                naive+=int(now);flagged+=int(now and not before);state[track_id]=now
                observations.append({'frame':number,'id':track_id,'bend':bend})
        fps.append(1/max(time.perf_counter()-begin,1e-9))
        if number in [0,12,24,35]:

images.append(cv2.cvtColor(result.plot(),cv2.COLOR_BGR2RGB));titles.append(f't={36+number/12:.2f}
c')
    cap.release()
    panel('pose_video_tracking.png',images,titles,cols=2)
    save('pose_video_tracking.json',{'tracks':len(state),'naive':naive,'flagged':flagged,
                'window_fps':float(np.mean(fps)),'observations':observations,'frames':36})
    print('Video text/image search and counters completed',flush=True)

if __name__=='__main__':
    main()
```

2.4. Полная программа контрольных опытов

Программа run_controls.py объединяет контроль человеческих ключевых точек, сверхразрешение и OCR. Входами служат bus.jpg, treponema_primary.jpg и контрольная текстовая строка. Изображения human_pose.png и mediapipe.png и таблица human_controls.json относятся к текущему пункту; результаты восстановления и распознавания текста анализируются в разделе 3.

Листинг — run_controls.py. Полный исходный текст файла run_controls.py.

```
"""Human pose controls, RRDB superresolution and OCR on Treponema material."""
from experiment_common import *
import sys
import subprocess
from PIL import Image, ImageDraw, ImageFont
from huggingface_hub import hf_hub_download
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

def main():
    from ultralytics import YOLO
    pose = YOLO(CACHE/'yolov8s-pose.pt')
    images, titles, observations = [], [], {}
    for name, path in [('people', LEGACY/'data/raw/bus.jpg'),
                       ('treponema', ROOT/'data/raw/treponema_primary.jpg')]:
        for threshold in ([.5] if name == 'people' else [.5, .25, .05]):
            result = pose(path, conf=threshold, imgsz=640, device='cpu', verbose=False)[0]
            observations[f'{name}_{threshold}'] = {'boxes':len(result.boxes), 'k':17}
            images.append(cv2.cvtColor(result.plot(), cv2.COLOR_BGR2RGB))
            titles.append(f'YOLOv8s-pose: {name}, conf={threshold}, N={len(result.boxes)}')
    panel('human_pose.png', images, titles, cols=2)
    import mediapipe as mp
    options = mp.tasks.vision.PoseLandmarkerOptions(

base_options=mp.tasks.BaseOptions(model_asset_path=str(CACHE/'pose_landmarker_lite.task')),
        num_poses=4)
    images, titles = [], []
    with mp.tasks.vision.PoseLandmarker.create_from_options(options) as detector:
        for name, path in [('people',LEGACY/'data/raw/bus.jpg'),
                           ('treponema',ROOT/'data/raw/treponema_primary.jpg')]:
            im = rgb(path)
            result = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=im))
            for person in result.pose_landmarks:
                for point in person:
                    cv2.circle(im, (round(point.x*im.shape[1]), round(point.y*im.shape[0])),
                               4, (255,100,0), -1)
            observations['mediapipe_'+name] = {'people':len(result.pose_landmarks), 'k':33}
            images.append(im)
            titles.append(f'MediaPipe: {name}; N={len(result.pose_landmarks)}')
    panel('mediapipe.png', images, titles, cols=2)
    save('human_controls.json', observations)

    sys.path.insert(0, str(LEGACY/'models'))
    from RRDBNet_arch import RRDBNet
    model = RRDBNet(3,3,64,23,gc=32)
    path = hf_hub_download('ai-forever/Real-ESRGAN','RealESRGAN_x4.pth')
    raw = torch.load(path, map_location='cpu', weights_only=True)
    mapping = {'body':'RRDB_trunk','conv_body':'trunk_conv',
               'conv_up1':'upconv1','conv_up2':'upconv2','conv_hr':'HRconv'}
    state = {'.'.join([mapping.get(k.split('.')[0], k.split('.')[0])] +
                      [p.upper() if p.startswith('rdb') else p for p in k.split('.')[1:]]):v
             for k,v in raw.items()}
    model.load_state_dict(state, strict=True)
    model.eval()
    reference = cv2.resize(rgb(ROOT/'data/raw/treponema_primary.jpg'), (384,240),
                           interpolation=cv2.INTER_AREA)
    small = cv2.resize(reference, (96,60), interpolation=cv2.INTER_AREA)
    tensor = torch.tensor(small.transpose(2,0,1).copy()).float()[None]/255
    with torch.no_grad():
        neural = np.round(model(tensor).clamp(0,1)[0].permute(1,2,0).numpy()*255).astype('uint8')
    cubic = cv2.resize(small, (384,240), interpolation=cv2.INTER_CUBIC)
    metrics = {}
    for name, im in [('Bicubic',cubic),('Real-ESRGAN',neural)]:
        metrics[name] = {'PSNR':float(peak_signal_noise_ratio(reference, im, data_range=255)),

'SSIM':float(structural_similarity(reference,im,channel_axis=2,data_range=255))}
    panel('superresolution_actual.png', [reference,cubic,neural], ['Эталон','Bicubic ×4','Real-
ESRGAN ×4'])
    save('superresolution_actual.json', metrics)

    # A known text label is intentionally drawn onto a microscopy illustration for the OCR
control.
    label = Image.new('RGB',(600,100),'white')
```

```
    font = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf',34)
    reference_text = 'Treponema pallidum 30 um'
    ImageDraw.Draw(label).text((12,25),reference_text,font=font,fill='black')
    label.save(OUT/'ocr_reference.png')
    small = cv2.resize(np.array(label), (150,25), interpolation=cv2.INTER_AREA)
    cubic = cv2.resize(small,(600,100),interpolation=cv2.INTER_CUBIC)
    tensor = torch.tensor(small.transpose(2,0,1).copy()).float()[None]/255
    with torch.no_grad():
        neural_text = np.round(model(tensor).clamp(0,1)
[0].permute(1,2,0).numpy()*255).astype('uint8')
    import easyocr
    reader = easyocr.Reader(['en'], gpu=False, model_storage_directory=str(CACHE/'easyocr'),
                            user_network_directory=str(CACHE/'easyocr/user'))
    def distance(a,b):
        previous = list(range(len(b)+1))
        for i,ca in enumerate(a,1):
            current = [i]
            for j,cb in enumerate(b,1):
                current.append(min(current[-1]+1,previous[j]+1,previous[j-1]+(ca!=cb)))
            previous=current
        return previous[-1]
    rows=[]
    for name, im in [('original',np.array(label)),('bicubic',cubic),('Real-ESRGAN',neural_text)]:
        file=OUT/f'ocr_{name}.png'
        Image.fromarray(im).save(file)
        tess=subprocess.run(['tesseract',str(file),'stdout','-l','eng','--psm','7'],
                            capture_output=True,text=True,check=True).stdout.strip()
        easy=' '.join(reader.readtext(im,detail=0))
        rows.append({'variant':name,'reference':reference_text,'Tesseract':tess,'EasyOCR':easy,
                     'CER_tesseract':distance(reference_text,tess)/len(reference_text),
                     'CER_easyocr':distance(reference_text,easy)/len(reference_text)})
    panel('ocr_comparison.png',[np.array(label),cubic,neural_text],
          ['Контрольная надпись','Уменьшение ×4 → Bicubic','Уменьшение ×4 → Real-ESRGAN'],cols=1)
    save('ocr_actual.json',rows)
    print('Completed human, SR and OCR controls', flush=True)

if __name__ == '__main__':
    main()
```

3. Трекинг, дисторсия, оптический поток, GAN и OCR

3.1. Сопровождение одного объекта

Вход — 36 последовательных кадров t07.ogv, начиная с 36-й секунды. Начальная рамка (113, 88, 44, 170) задана в пикселях кадра 360 × 288 и охватывает вытянутый видимый объект. Все четыре трекера получают одинаковый первый кадр и рамку. Каждый выполняет 35 обновлений; время измеряется непосредственно вокруг вызова update, без декодирования видео и отрисовки.

Алгоритм выполнения. 1) Прочитать заданный отрезок. 2) Создать MIL, MOSSE, KCF и CSRT через cv2.legacy. 3) Инициализировать каждый трекер одной рамкой. 4) Для следующих кадров сохранить флаг ok, рамку и время. 5) Вычислить долю ok и FPS. 6) Сопоставить рамки на одинаковых моментах видео.

Таблица 5

Трекер	**Обновлений**	**Доля ok**	**FPS**
MIL	35	1.000	35.18
MOSSE	35	0.000	5686.07
KCF	35	1.000	489.07
CSRT	35	1.000	47.21

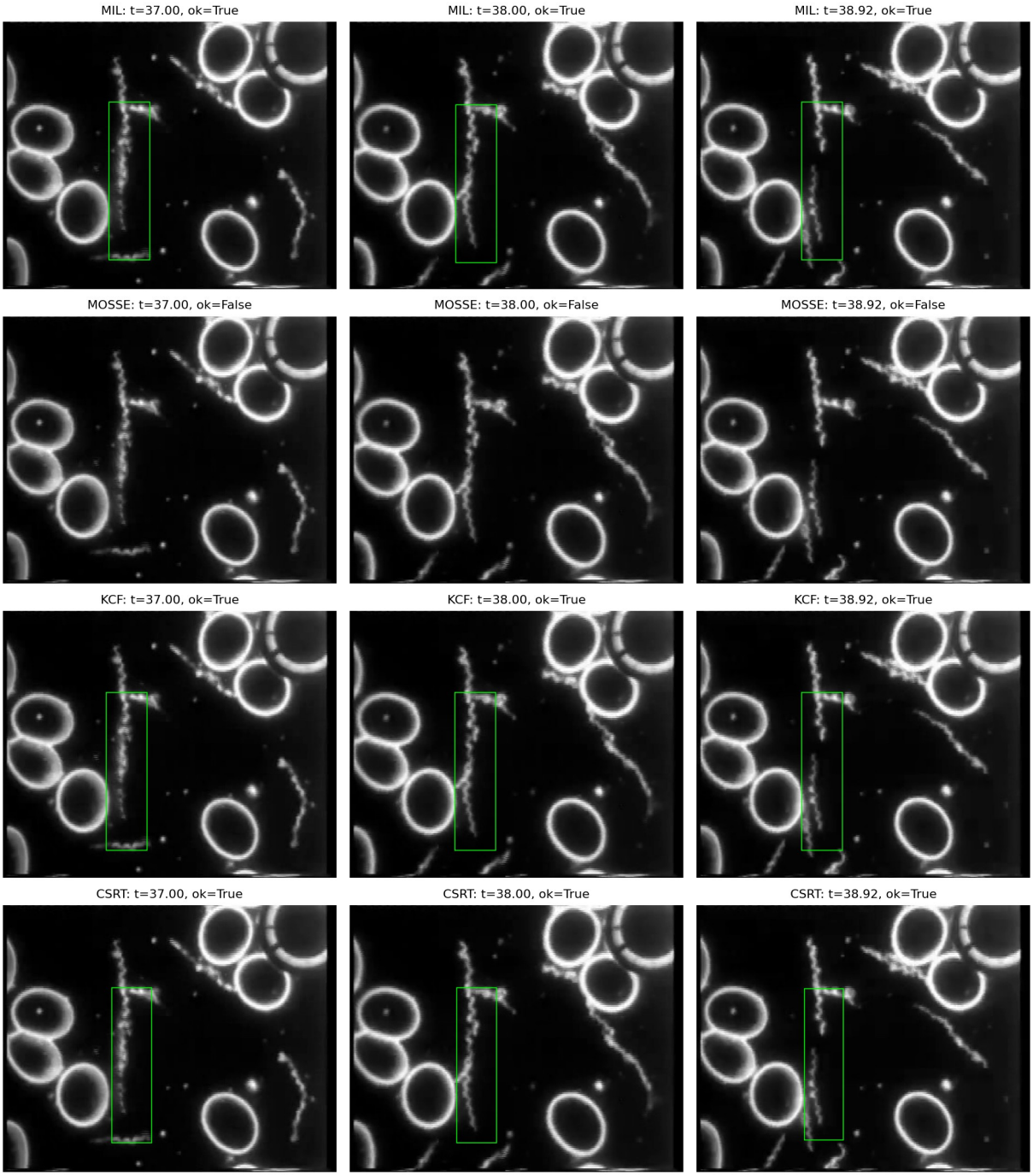


Рисунок 6 — MIL, MOSSE, KCF и CSRT на одинаковых кадрах микроскопического видео

MIL, KCF и CSRT возвращают рамку во всех 35 обновлениях. Для MOSSE доля ok равна 0; его высокая скорость в этой серии характеризует короткие вызовы без принятой позиции. KCF выполняет обновления быстрее MIL и CSRT. Флаг ok отражает внутреннее решение алгоритма, поэтому оценка устойчивости дополняется визуальным просмотром положения рамки. Здесь доля ok приводится отдельно от IoU: это разные характеристики.

MIL использует онлайн-обучение по наборам положительных и отрицательных примеров. MOSSE строит корреляционный фильтр, KCF ускоряет корреляционный поиск за счёт структуры обучающих примеров, CSRT использует пространственную и канальную надёжность. При похожих соседних объектах обновляемая модель может постепенно сместиться на соседний фрагмент. Скорость и точность поэтому оцениваются совместно.

3.2. Контроль IoU по размеченным кадрам

Дополнительно сохранён контроль SOT на отдельном микроскопическом видео бактерий из сравнительного набора. Для пяти кадров после инициализации заданы опорные рамки. На них вычислен IoU = площадь пересечения / площадь объединения. Этот контроль дополняет опыт на трепонеме примером оценки геометрической точности по явной разметке.

Таблица 6

Трекер	**Средний IoU**	**Доля IoU ≥ 0,5**	**Опорных кадров**
MIL	0.6646	0.80	5
KCF	0.0000	0.00	5
CSRT	0.8483	1.00	5

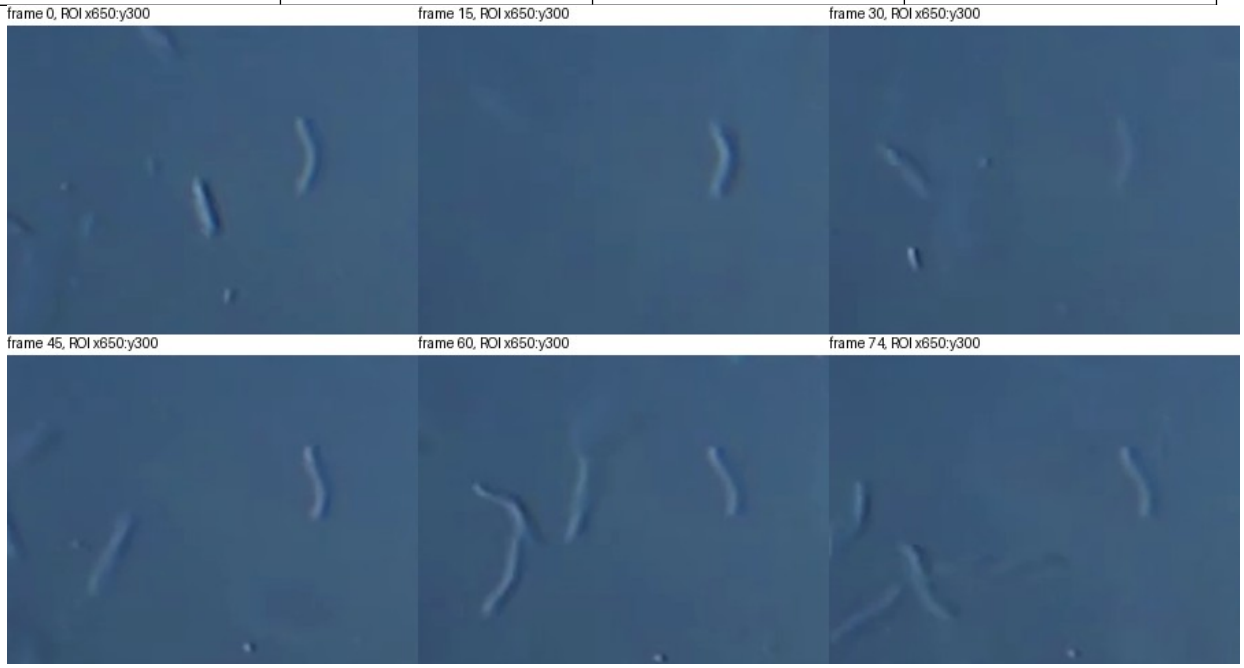


Рисунок 7 — Опорные рамки для контроля качества сопровождения на сравнительном видео

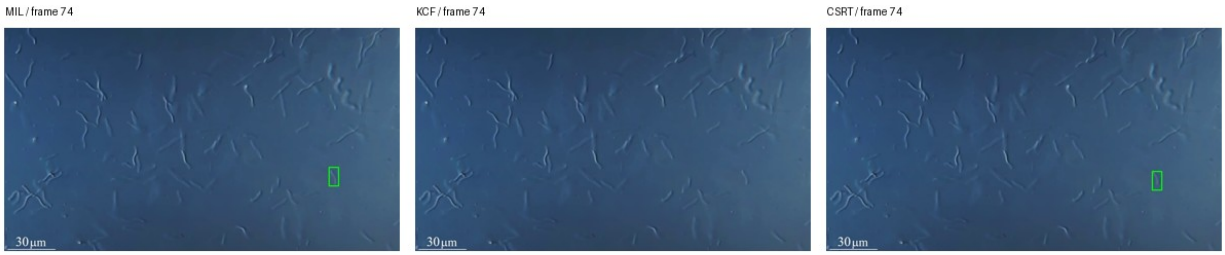


Рисунок 8 — Контрольное сравнение рамок MIL, KCF и CSRT

На этом отрезке наибольший средний IoU имеет CSRT: 0,8483. MIL даёт 0,6646. Оценка вычислена на пяти выбранных кадрах, поэтому характеризует контрольные моменты данного ролика. Результаты сравнительного видео и t07.ogv рассматриваются раздельно, поскольку у них различаются объекты, контраст и движение.

3.3. Искусственная дисторсия и восстановление

Вход — основная иллюстрация, приведённая к 435 × 270. Известная радиальная дисторсия задаётся коэффициентами $k_1 = -0{,}25$ и $k_2 = 0{,}05$. В нормированных координатах множитель радиального изменения равен $1 + k_1 r^2 + k_2 r^4$. После внесения искажения выполняются коррекция истинными коэффициентами и подбор коэффициентов по максимуму PSNR методом Нелдера — Мида.

Алгоритм выполнения. 1) Задать матрицу камеры K. 2) Построить карту искусственного искажения. 3) Получить distorted через remap. 4) Определить целевую функцию −PSNR. 5) Запустить Nelder–Mead с тремя начальными вершинами. 6) Сохранить коэффициенты, PSNR и изображения до и после коррекции.

Таблица 7

Величина	**Результат**
Истинные k_1; k_2	−0,25; 0,05
Подобранные k_1; k_2	-0.252298; 0.045958
Итераций	56
PSNR искажённого, дБ	11.1047
PSNR коррекции истинными k, дБ	30.4079
PSNR подобранной коррекции, дБ	31.0304



Рисунок 9 — Исходник, искусственная радиальная дисторсия и подобранная коррекция

После коррекции PSNR повышается с 11,10 до 31,03 дБ. Найденные коэффициенты близки к заданным. Небольшое отличие объясняется дискретизацией карты и интерполяцией пикселей: оптимум сравнения готовых растров может отличаться от исходных непрерывных коэффициентов. Это контролируемая проверка алгоритма на известном преобразовании.

3.4. Оптический поток

Алгоритм выполнения. 1) Взять два соседних кадра t07.ogv. 2) Перевести их в оттенки серого. 3) Рассчитать плотный поток Фарнебэка. 4) Выделить до 100 углов и проследить их методом Лукаса — Канаде. 5) Усреднить компоненты плотного потока в начальной ROI. 6) Умножить длину среднего вектора на частоту 12 кадров/с.

Таблица 8

Выходная величина	**Значение**
Средний вектор, px/кадр	1.780937, -0.725222
Скорость среднего вектора, px/c	23.0752
Точек LK обнаружено / принято	100 / 100

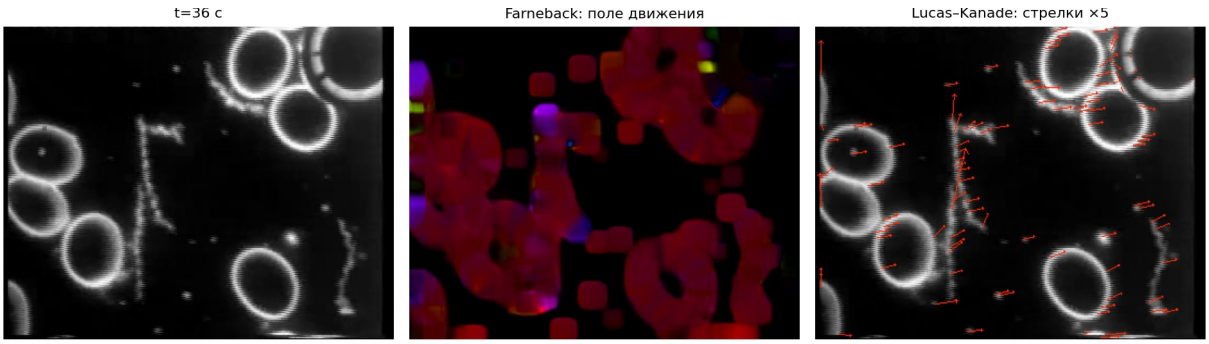


Рисунок 10 — Плотный поток Фарнебэка и разреженный поток Лукаса — Канаде; стрелки увеличены в 5 раз

Полученный средний вектор (1,78094; −0,72522) px/кадр соответствует движению вправо и вверх в координатах кадра. Скорость среднего вектора равна 23,0752 px/c. Усредняются компоненты векторов, а не их углы. В область включены объект и окружающий фон, поэтому значение описывает выбранную ROI. Вызов LK принял 100 из 100 точек по своему статусу; дополнительная проверка обратным проходом является отдельным критерием качества.

3.5. Сверхразрешение и распознавание текста

Для контролируемого сравнения иллюстрация приводится к 384 × 240, уменьшается до 96 × 60 и восстанавливается в четыре раза. Бикубическая интерполяция сравнивается с Real-ESRGAN на архитектуре RRDBNet. Эталоном служит изображение перед уменьшением. PSNR оценивает пиксельную ошибку, SSIM — структурное сходство; обе метрики вычисляются на одинаковом размере и диапазоне 0–255.

Алгоритм выполнения. 1) Сформировать эталон и уменьшенный вход. 2) Выполнить Bicubic ×4. 3) Загрузить веса Real-ESRGAN в RRDBNet и выполнить прямой проход. 4) Ограничить выход диапазоном [0, 1] и перевести в uint8. 5) Рассчитать PSNR и SSIM. 6) Для OCR отдельно сформировать строку с известным эталоном, уменьшить её и прочитать восстановленный вариант.

Таблица 9

Метод	**PSNR, дБ**	**SSIM**
Bicubic	24.8671	0.666393
Real-ESRGAN	23.9367	0.616343

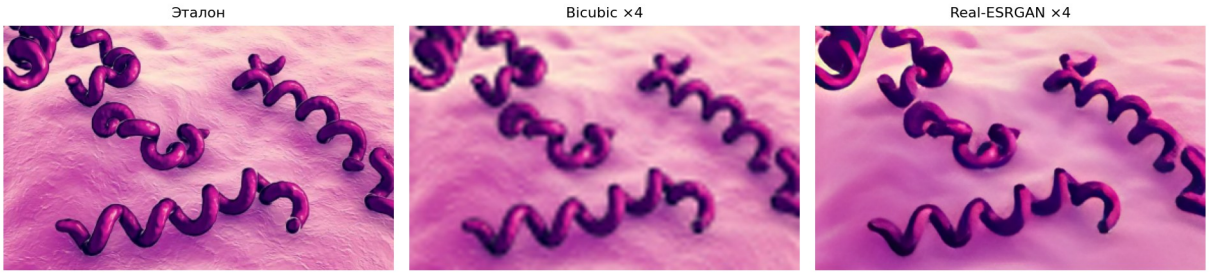


Рисунок 11 — Эталон и результаты восстановления изображения трепонемы двумя методами

Bicubic получает PSNR 24,8671 дБ и SSIM 0,666393, Real-ESRGAN — 23,9367 дБ и 0,616343. На выбранной иллюстрации интерполяция ближе к эталону по обеим метрикам. Нейросетевая реконструкция меняет локальную текстуру, поэтому визуальная резкость оценивается отдельно от точности восстановления исходных пикселей.

Для OCR используется контрольная надпись «Treponema pallidum 30 um», сформированная программно шрифтом DejaVu Sans. Это текстовый тест с известной правильной строкой. Изображение надписи 600 × 100 уменьшается до 150 × 25 и восстанавливается. CER вычисляется как расстояние Левенштейна между распознанной и эталонной строками, делённое на длину эталона; регистр и пробелы учитываются.

Таблица 10

Вариант	**Tesseract / CER**	**EasyOCR / CER**
original	Treponema pallidum 30 um / 0.000	Treponema pallidum 30 um / 0.000
bicubic	Treponema pallidum 30 um / 0.000	Treponem & pallidui 30 Uin / 0.250



Рисунок 12 — Контрольная надпись и её восстановление для проверки OCR

На исходной надписи оба распознавателя возвращают эталонную строку с CER = 0. После уменьшения и бикубического восстановления Tesseract сохраняет CER = 0, EasyOCR даёт CER = 0,25. Выходные строки в таблице позволяют оценить конкретные замены и вставки символов, а не только итоговую метрику.



Рисунок 13 — Дополнительный контроль OCR на подписи масштабной линейки из сравнительного набора

На сохранённом контрольном фрагменте масштабной линейки Tesseract возвращает «30 um» для цветного изображения и «30um» после пороговой обработки. Это показывает влияние пробелов и обозначения единицы измерения на буквальное сравнение строк. Подпись этого

контрольного изображения не переносится на масштаб основной иллюстрации трепонемы.

3.6. Фильтрация и выделение движущегося переднего плана

Фильтр Гаусса 5 × 5 сглаживает высокочастотные изменения, медианный фильтр 5 × 5 подавляет отдельные выбросы, Canny выделяет контуры при порогах 60 и 140. Для видео MOG2 формирует модель фона с history = 50, varThreshold = 16 и learningRate = 0,02. Маска MOG2 описывает изменение относительно фона, а цветовая маска — выбранные цвета изображения.

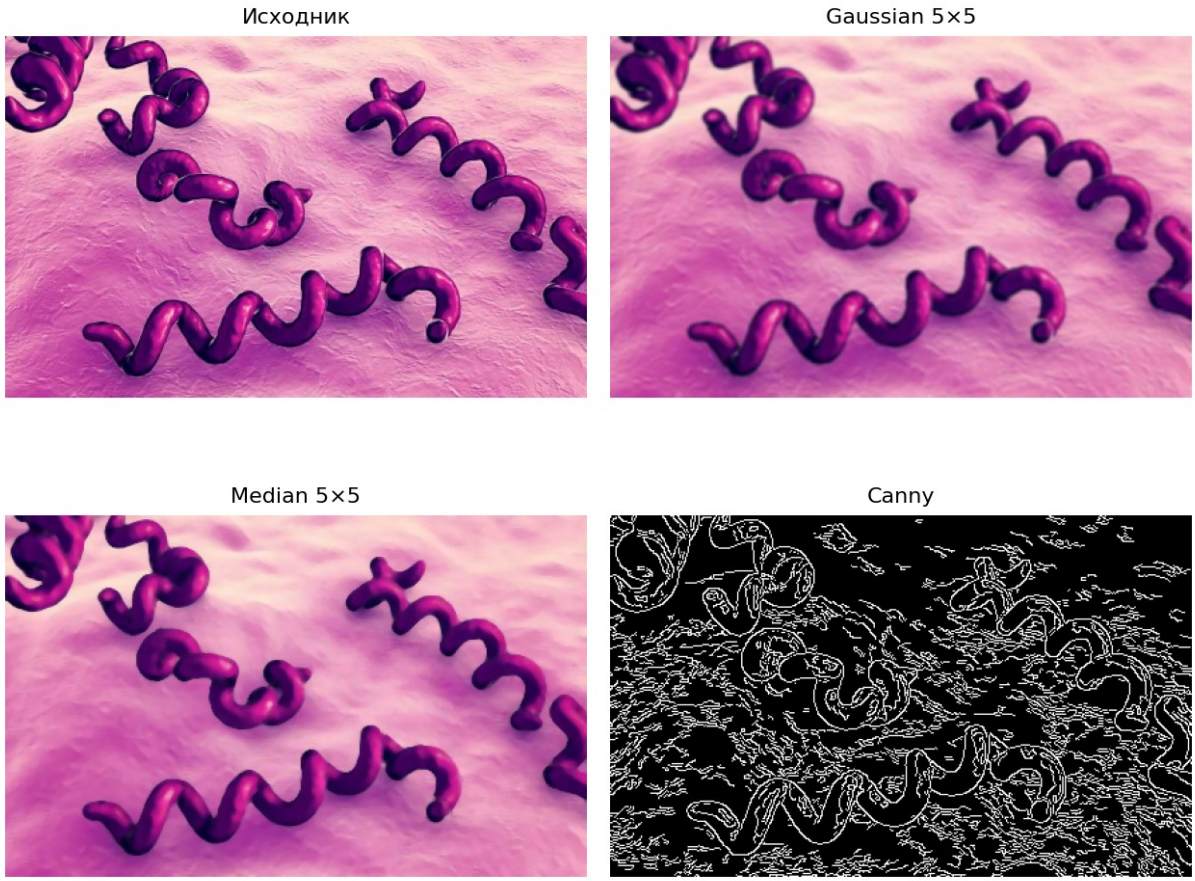


Рисунок 14 — Исходное изображение, гауссово и медианное сглаживание, границы Canny

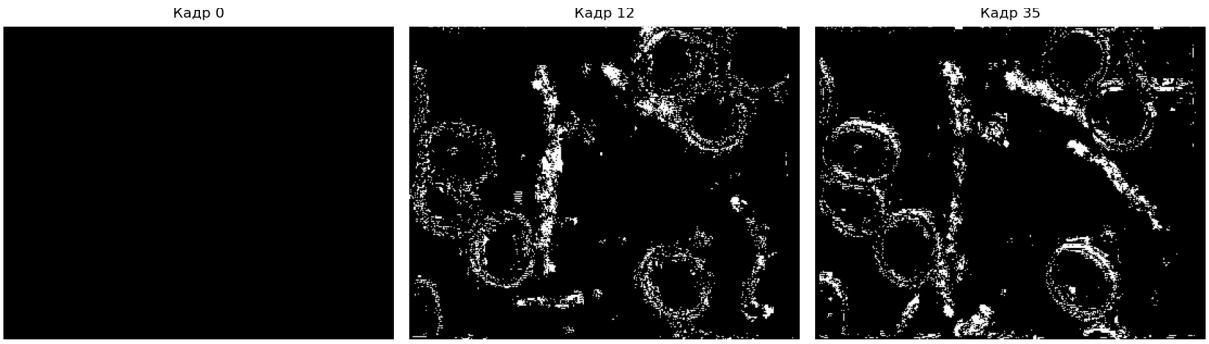


Рисунок 15 — Маски MOG2 на кадрах 0, 12 и 35 одного отрезка видео

Таблица 11

Кадр	**Доля переднего плана**
0	1.000000
12	0.064226
35	0.073187

Первый кадр целиком относится к переднему плану при инициализации модели. К кадру 12 доля изменившихся пикселей равна 0,06423, к кадру 35 — 0,07319. Эти доли характеризуют динамику модели фона и не используются как подсчёт бактерий.

3.7. Полные программы обработки

Листинг — run_geometry.py. Полный исходный текст файла run_geometry.py.

```
"""Distortion, tracking, optical flow and image filtering on local Treponema data."""
from experiment_common import *
import time
from scipy.optimize import minimize
from skimage.metrics import peak_signal_noise_ratio as psnr

def main():
    source = cv2.resize(rgb(ROOT/'data/raw/treponema_primary.jpg'),(435,270))
    h,w=source.shape[:2]
    K=np.array([[.9*w,0,w/2],[0,.9*w,h/2],[0,0,1]],np.float64)
    D=np.array([-.25,.05,0,0,0],np.float64)
    xx,yy=np.meshgrid(np.arange(w),np.arange(h))
    pts=np.stack([xx,yy],-1).reshape(-1,1,2).astype(np.float32)
    coords=cv2.undistortPointsIter(pts,K,D,None,K,
              (cv2.TERM_CRITERIA_COUNT|cv2.TERM_CRITERIA_EPS,40,1e-9)).reshape(h,w,2)
    distorted=cv2.remap(source,coords[:,:,0],coords[:,:,1],cv2.INTER_LINEAR)
    def correct(values):
        return cv2.undistort(distorted,K,np.array([*values,0,0,0],np.float64))
    optimum=minimize(lambda v:-psnr(source,correct(v),data_range=255),[0.,0.],
                     method='Nelder-Mead',options={'initial_simplex':np.array([[0,0],[-.3,0],
[0,.1]]),
                     'maxiter':300,'xatol':1e-5,'fatol':1e-6})
    panel('distortion.png',[source,distorted,correct(optimum.x)],
          ['Исходное изображение','Искусственная дисторсия','Подобранная коррекция'])
    save('distortion.json',{'true':[-.25,.05],'estimated':optimum.x.tolist(),
         'PSNR_distorted':float(psnr(source,distorted,data_range=255)),
         'PSNR_true':float(psnr(source,correct([-.25,.05]),data_range=255)),
         'PSNR_optimized':float(-optimum.fun),'iterations':int(optimum.nit)})
    panel('filters.png',[source,cv2.GaussianBlur(source,(5,5),0),cv2.medianBlur(source,5),
```

```
          cv2.Canny(cv2.cvtColor(source,cv2.COLOR_RGB2GRAY),60,140)],
          ['Исходник','Gaussian 5×5','Median 5×5','Canny'],cols=2)
    cap=cv2.VideoCapture(str(ROOT/'data/raw/t07.ogv'))
    cap.set(cv2.CAP_PROP_POS_MSEC,36000)
    frames=[]
    for _ in range(36):
        ok,frame=cap.read()
        if ok:frames.append(frame)
    cap.release()
    box=(113,88,44,170)  # Visible vertical spiral at t=36 s, coordinates in the 360×288 frame.
    first=frames[0]
    factories={'MIL':cv2.legacy.TrackerMIL_create,'MOSSE':cv2.legacy.TrackerMOSSE_create,
               'KCF':cv2.legacy.TrackerKCF_create,'CSRT':cv2.legacy.TrackerCSRT_create}
    metrics={}; visuals=[]; titles=[]; observations=[]
    for name,factory in factories.items():
        tracker=factory();tracker.init(first,box)
        times=[]; found=[]; boxes=[]
        for i,frame in enumerate(frames[1:],1):

begin=time.perf_counter();ok,bb=tracker.update(frame);times.append(time.perf_counter()-begin)
            found.append(bool(ok));boxes.append(list(map(float,bb)))
            if i in [12,24,35]:
                shown=frame.copy()
                if ok:
                    x,y,w,h=map(int,bb);cv2.rectangle(shown,(x,y),(x+w,y+h),(0,255,0),1)
                visuals.append(cv2.cvtColor(shown,cv2.COLOR_BGR2RGB));titles.append(f'{name}:
t={36+i/12:.2f}, ok={ok}')
        metrics[name]={'FPS':len(times)/sum(times),'valid_fraction':float(np.mean(found)),
                       'updates':len(times)}
        observations.append({'tracker':name,'boxes':boxes,'valid':found})
    panel('tracking_actual.png',visuals,titles,cols=3)
    save('tracking_actual.json',{'video':'t07.ogv','start_s':36,'initial_box':box,
          'metrics':metrics,'observations':observations})
    a,b=[cv2.cvtColor(f,cv2.COLOR_BGR2GRAY) for f in frames[:2]]
    flow=cv2.calcOpticalFlowFarneback(a,b,None,.5,3,15,3,5,1.2,0)
    mag,angle=cv2.cartToPolar(flow[:,:,0],flow[:,:,1]);hsv=np.zeros_like(first)

hsv[:,:,0]=angle*90/np.pi;hsv[:,:,1]=255;hsv[:,:,2]=cv2.normalize(mag,None,0,255,cv2.NORM_MINMAX)
    dense=cv2.cvtColor(hsv,cv2.COLOR_HSV2RGB)
    points=cv2.goodFeaturesToTrack(a,100,.01,5)
    next_points,status,_=cv2.calcOpticalFlowPyrLK(a,b,points,None)
    drawn=cv2.cvtColor(first,cv2.COLOR_BGR2RGB)
    for p,q in zip(points[status.ravel()==1],next_points[status.ravel()==1]):
        cv2.arrowedLine(drawn,tuple(p[0].astype(int)),tuple((p[0]+5*(q[0]-p[0])).astype(int)),
(255,40,0),1)
    panel('flow_actual.png',[cv2.cvtColor(first,cv2.COLOR_BGR2RGB),dense,drawn],
          ['t=36 с','Farneback: поле движения','Lucas–Kanade: стрелки ×5'])
    x,y,w,h=box;v=flow[y:y+h,x:x+w].mean(axis=(0,1))
    save('flow_actual.json',
{'mean_vector_px':v.tolist(),'speed_px_s':float(np.linalg.norm(v)*12),
                           'LK_detected':len(points),'LK_accepted':int(status.sum())})
    mog=cv2.createBackgroundSubtractorMOG2(history=50,varThreshold=16,detectShadows=False)
    images=[];titles=[];fractions={}
    for i,frame in enumerate(frames):
        mask=mog.apply(frame,learningRate=.02)
        if i in [0,12,35]:
            images.append(mask);titles.append(f'Кадр
{i}');fractions[str(i)]=float((mask>0).mean())
    panel('foreground.png',images,titles)
    save('foreground.json',fractions)
    print('Completed geometry and video experiments',flush=True)

if __name__ == '__main__':
    main()
```

Листинг — Полная программа сравнительного контроля IoU и OCR. Полный исходный текст файла vision_baselines.py.

```
from pathlib import Path
import os,json,time,subprocess,csv
import cv2,numpy as np
import matplotlib;matplotlib.use('Agg')
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio,structural_similarity
```

```
from scipy.signal import butter,sosfilt
from scipy.io import wavfile
P=Path(__file__).resolve().parents[1];R=P/'results';cv2.setNumThreads(2)
metrics={};im=cv2.imread(str(P/'data/raw/s06_640.png'));gray=cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)
variants={'Original':im,'Gaussian 5x5':cv2.GaussianBlur(im,(5,5),0),'Median
5x5':cv2.medianBlur(im,5),'Canny':cv2.Canny(gray,60,150)}
fig,ax=plt.subplots(2,2,figsize=(10,7))
for a,(n,v) in zip(ax.ravel(),variants.items()):a.imshow(cv2.cvtColor(v,cv2.COLOR_BGR2RGB) if
v.ndim==3 else v,cmap='gray');a.set_title(n);a.axis('off')
fig.tight_layout();fig.savefig(R/'filters.png',dpi=150);plt.close(fig)
hsv=cv2.cvtColor(im,cv2.COLOR_BGR2HSV);mask=cv2.inRange(hsv,(100,70,30),
(155,255,255));mask=cv2.morphologyEx(mask,cv2.MORPH_OPEN,np.ones((3,3),np.uint8));cv2.imwrite(str
(R/'color_mask.png'),mask)
n,lab,stats,centroids=cv2.connectedComponentsWithStats(mask)
metrics['color_segmentation']={'foreground_fraction':float((mask>0).mean()),'components_over_100p
x':int((stats[1:,4]>100).sum()),'note':'Connected regions merge touching cells; not a bacteria
count.'}
# Downsample then reconstruct to evaluate information loss against a genuine reference.
ref=im[64:320,128:384];small=cv2.resize(ref,(64,64),interpolation=cv2.INTER_AREA);scores={}
fig,ax=plt.subplots(1,4,figsize=(12,3));ax[0].imshow(cv2.cvtColor(ref,cv2.COLOR_BGR2RGB));ax[0].s
et_title('Reference')
for a,(name,method) in zip(ax[1:],[('Nearest',cv2.INTER_NEAREST),('Bilinear',cv2.INTER_LINEAR),
('Bicubic',cv2.INTER_CUBIC)]):
 out=cv2.resize(small,
(256,256),interpolation=method);scores[name]={'PSNR':float(peak_signal_noise_ratio(ref,out,data_r
ange=255)),'SSIM':float(structural_similarity(ref,out,channel_axis=2,data_range=255))};a.imshow(c
v2.cvtColor(out,cv2.COLOR_BGR2RGB));a.set_title(f'{name}\nPSNR={scores[name]["PSNR"]:.2f}')
for a in ax:a.axis('off')
fig.tight_layout();fig.savefig(R/'resolution.png',dpi=150);plt.close(fig);metrics['resolution']=s
cores
cv2.imwrite(str(R/'sr_reference.png'),ref);cv2.imwrite(str(R/'sr_lowres.png'),small)
cap=cv2.VideoCapture(str(P/'data/raw/bacteria_12s.mp4'));frames=[]
while len(frames)<75:
 ok,f=cap.read()
 if not ok:break
 frames.append(f)
cap.release();h,w=frames[0].shape[:2]
# Sparse manual boxes around one visually persistent curved bacterium.
gt={0:(785,342,23,45),15:(787,341,25,44),30:(784,340,25,44),45:(789,343,24,43),60:
(787,345,26,45),74:(788,347,26,45)}
def iou(a,b):
 x,y,ww,hh=a;xx,yy,www,hhh=b;inter=max(0,min(x+ww,xx+www)-max(x,xx))*max(0,min(y+hh,yy+hhh)-
max(y,yy));u=ww*hh+www*hhh-inter;return inter/u if u>0 else 0
track={};allrows=[]
for name,ctor in [('MIL',cv2.TrackerMIL_create),('KCF',cv2.TrackerKCF_create),
('CSRT',cv2.TrackerCSRT_create)]:
 tracker=ctor();tracker.init(frames[0],gt[0]);boxes=[gt[0]];oklist=[];times=[]
 for fi,frame in enumerate(frames[1:],1):
  t=time.perf_counter();ok,b=tracker.update(frame);times.append(time.perf_counter()-
t);oklist.append(bool(ok));boxes.append(tuple(b) if ok else None)

allrows.append({'tracker':name,'frame':fi,'seconds':fi/25,'ok':int(ok),'x':float(b[0]),'y':float(
b[1]),'w':float(b[2]),'h':float(b[3])})
 vals=[iou(boxes[f],b) if boxes[f] else 0 for f,b in gt.items() if f>0]

track[name]={'mean_IoU_sparse':float(np.mean(vals)),'success_IoU_05_sparse':float(np.mean(np.arra
y(vals)>.5)),'valid_fraction':float(np.mean(oklist)),'FPS':1/
float(np.mean(times)),'reference_frames_excluding_init':5,'ious':vals}
 print(name,track[name],flush=True)
 for fi in [0,30,74]:
  f=frames[fi].copy();b=boxes[fi]
  if b:
   x,y,bw,bh=map(int,b);cv2.rectangle(f,(x,y),(x+bw,y+bh),(0,255,0),2)
  cv2.imwrite(str(R/f'track_{name}_{fi}.jpg'),f)
metrics['tracking']=track
with (R/'tracking.csv').open('w') as f:
 wr=csv.DictWriter(f,fieldnames=allrows[0].keys());wr.writeheader();wr.writerows(allrows)
(R/'tracking_reference.json').write_text(json.dumps(gt,indent=2))
p=cv2.cvtColor(frames[0],cv2.COLOR_BGR2GRAY);n=cv2.cvtColor(frames[1],cv2.COLOR_BGR2GRAY)
flow=cv2.calcOpticalFlowFarneback(p,n,None,.5,3,15,3,5,1.2,0)
roi=np.zeros_like(p);roi[342:387,785:808]=255
p0=cv2.goodFeaturesToTrack(p,120,.01,8,mask=roi);sparse={}
if p0 is not None:

p1,st,_=cv2.calcOpticalFlowPyrLK(p,n,p0,None);pback,stback,_=cv2.calcOpticalFlowPyrLK(n,p,p1,None
);ok=(st.ravel()==1)&(stback.ravel()==1)&(np.linalg.norm((pback-
p0).reshape(-1,2),axis=1)<1);d=(p1-p0).reshape(-1,2)[ok]
```

```
sparse={'points_detected':len(p0),'points_accepted':int(ok.sum()),'mean_vector_px':d.mean(0).toli
st() if len(d) else None}
mean=flow[roi>0].mean(0);metrics['flow']={'Farneback_mean_vector_px':mean.tolist(),'ROI_speed_px_
s':float(np.linalg.norm(mean)*25),'Lucas_Kanade':sparse,'note':'No dense ground-truth flow; ROI
includes local background.'}
mag,ang=cv2.cartToPolar(flow[:,:,0],flow[:,:,1]);vis=np.zeros_like(frames[0]);vis[:,:,0]=ang*180/
np.pi/
2;vis[:,:,1]=255;vis[:,:,2]=cv2.normalize(mag,None,0,255,cv2.NORM_MINMAX);cv2.imwrite(str(R/'flow
.png'),cv2.cvtColor(vis,cv2.COLOR_HSV2BGR))
f=frames[0].copy()
for yy in range(20,h-30,20):
 for xx in range(20,w-20,20):
  dx,dy=flow[yy,xx];cv2.arrowedLine(f,(xx,yy),(round(xx+dx*12),round(yy+dy*12)),
(0,255,0),1,tipLength=.3)
cv2.imwrite(str(R/'flow_arrows.jpg'),f)
# Real text from original video scale bar; no invented OCR observation.
ocr=frames[0]
[492:536,10:115];ocr=cv2.resize(ocr,None,fx=4,fy=4,interpolation=cv2.INTER_CUBIC);cv2.imwrite(str
(R/'ocr_input.png'),ocr)
ocr_rows=[]
for variant in ['color','threshold']:
 a=ocr if variant=='color' else
cv2.threshold(cv2.cvtColor(ocr,cv2.COLOR_BGR2GRAY),170,255,cv2.THRESH_BINARY)[1];f=R/
f'ocr_{variant}.png';cv2.imwrite(str(f),a)
 r=subprocess.run(['tesseract',str(f),'stdout','--psm','7','-
l','eng'],capture_output=True,text=True);ocr_rows.append({'variant':variant,'returncode':r.return
code,'text':r.stdout.strip(),'reference':'30 µm'})
metrics['OCR']=ocr_rows
back=cv2.createBackgroundSubtractorMOG2(history=50,varThreshold=16,detectShadows=False);fractions
=[]
for i,f in enumerate(frames):

m=back.apply(f,learningRate=.02);m=cv2.morphologyEx(m,cv2.MORPH_OPEN,np.ones((3,3),np.uint8));fra
ctions.append(float((m>0).mean()))
 if i in [0,25,74]:cv2.imwrite(str(R/f'mog2_{i}.png'),m)
metrics['MOG2']={'foreground_fraction_frame0':fractions[0],'foreground_fraction_frame25':fraction
s[25],'foreground_fraction_frame74':fractions[74],'note':'Motion mask, not semantic
segmentation.'}
# Explicitly synthetic audio control: the microscopy video has no audio track.
sr=16000;t=np.arange(sr)/sr;clean=np.sin(2*np.pi*440*t);noise=.4*np.sin(2*np.pi*4000*t);sig=clean
+noise;out=sosfilt(butter(4,1200,fs=sr,output='sos'),sig)
wavfile.write(str(R/'audio_synthetic_before.wav'),sr,(sig/1.4*.9*32767).astype('int16'));wavfile.
write(str(R/'audio_synthetic_after.wav'),sr,(out/1.4*.9*32767).astype('int16'))
freq=np.fft.rfftfreq(sr,1/sr);fig,ax=plt.subplots(figsize=(8,3));ax.plot(freq,20*np.log10(np.maxi
mum(abs(np.fft.rfft(sig)),1e-8)),label='Before');ax.plot(freq,20*np.log10(np.maximum(abs(np.fft.r
fft(out)),1e-8)),label='Low-pass 1200
Hz');ax.set(xlim=(0,5000),ylim=(-30,90),xlabel='Hz',ylabel='Magnitude,
dB');ax.legend();fig.tight_layout();fig.savefig(R/'audio_filter.png',dpi=150);plt.close(fig)
metrics['audio']={'source':'synthetic two-tone control, not bacteria video
audio','sample_rate':sr,'signal_Hz':440,'noise_Hz':4000,'cutoff_Hz':1200}
(R/'baselines.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2));print(json.dumps
(metrics,ensure_ascii=False,indent=2),flush=True)
```

Программа сверхразрешения и OCR приведена целиком также в этом разделе для самостоятельного просмотра всех операций восстановления. Определение архитектуры RRDBNet включено следом; служебные функции находятся в листинге experiment_common.py раздела 1.

Листинг — run_controls.py. Полный исходный текст файла run_controls.py.

```
"""Human pose controls, RRDB superresolution and OCR on Treponema material."""
from experiment_common import *
import sys
import subprocess
from PIL import Image, ImageDraw, ImageFont
from huggingface_hub import hf_hub_download
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
```

```
def main():
    from ultralytics import YOLO
    pose = YOLO(CACHE/'yolov8s-pose.pt')
    images, titles, observations = [], [], {}
    for name, path in [('people', LEGACY/'data/raw/bus.jpg'),
                       ('treponema', ROOT/'data/raw/treponema_primary.jpg')]:
        for threshold in ([.5] if name == 'people' else [.5, .25, .05]):
            result = pose(path, conf=threshold, imgsz=640, device='cpu', verbose=False)[0]
            observations[f'{name}_{threshold}'] = {'boxes':len(result.boxes), 'k':17}
            images.append(cv2.cvtColor(result.plot(), cv2.COLOR_BGR2RGB))
            titles.append(f'YOLOv8s-pose: {name}, conf={threshold}, N={len(result.boxes)}')
    panel('human_pose.png', images, titles, cols=2)
    import mediapipe as mp
    options = mp.tasks.vision.PoseLandmarkerOptions(

base_options=mp.tasks.BaseOptions(model_asset_path=str(CACHE/'pose_landmarker_lite.task')),
        num_poses=4)
    images, titles = [], []
    with mp.tasks.vision.PoseLandmarker.create_from_options(options) as detector:
        for name, path in [('people',LEGACY/'data/raw/bus.jpg'),
                           ('treponema',ROOT/'data/raw/treponema_primary.jpg')]:
            im = rgb(path)
            result = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=im))
            for person in result.pose_landmarks:
                for point in person:
                    cv2.circle(im, (round(point.x*im.shape[1]), round(point.y*im.shape[0])),
                               4, (255,100,0), -1)
            observations['mediapipe_'+name] = {'people':len(result.pose_landmarks), 'k':33}
            images.append(im)
            titles.append(f'MediaPipe: {name}; N={len(result.pose_landmarks)}')
    panel('mediapipe.png', images, titles, cols=2)
    save('human_controls.json', observations)

    sys.path.insert(0, str(LEGACY/'models'))
    from RRDBNet_arch import RRDBNet
    model = RRDBNet(3,3,64,23,gc=32)
    path = hf_hub_download('ai-forever/Real-ESRGAN','RealESRGAN_x4.pth')
    raw = torch.load(path, map_location='cpu', weights_only=True)
    mapping = {'body':'RRDB_trunk','conv_body':'trunk_conv',
               'conv_up1':'upconv1','conv_up2':'upconv2','conv_hr':'HRconv'}
    state = {'.'.join([mapping.get(k.split('.')[0], k.split('.')[0])] +
                      [p.upper() if p.startswith('rdb') else p for p in k.split('.')[1:]]):v
             for k,v in raw.items()}
    model.load_state_dict(state, strict=True)
    model.eval()
    reference = cv2.resize(rgb(ROOT/'data/raw/treponema_primary.jpg'), (384,240),
                           interpolation=cv2.INTER_AREA)
    small = cv2.resize(reference, (96,60), interpolation=cv2.INTER_AREA)
    tensor = torch.tensor(small.transpose(2,0,1).copy()).float()[None]/255
    with torch.no_grad():
        neural = np.round(model(tensor).clamp(0,1)[0].permute(1,2,0).numpy()*255).astype('uint8')
    cubic = cv2.resize(small, (384,240), interpolation=cv2.INTER_CUBIC)
    metrics = {}
    for name, im in [('Bicubic',cubic),('Real-ESRGAN',neural)]:
        metrics[name] = {'PSNR':float(peak_signal_noise_ratio(reference, im, data_range=255)),

'SSIM':float(structural_similarity(reference,im,channel_axis=2,data_range=255))}
    panel('superresolution_actual.png', [reference,cubic,neural], ['Эталон','Bicubic ×4','Real-
ESRGAN ×4'])
    save('superresolution_actual.json', metrics)

    # A known text label is intentionally drawn onto a microscopy illustration for the OCR
control.
    label = Image.new('RGB',(600,100),'white')
    font = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf',34)
    reference_text = 'Treponema pallidum 30 um'
    ImageDraw.Draw(label).text((12,25),reference_text,font=font,fill='black')
    label.save(OUT/'ocr_reference.png')
    small = cv2.resize(np.array(label), (150,25), interpolation=cv2.INTER_AREA)
    cubic = cv2.resize(small,(600,100),interpolation=cv2.INTER_CUBIC)
    tensor = torch.tensor(small.transpose(2,0,1).copy()).float()[None]/255
    with torch.no_grad():
        neural_text = np.round(model(tensor).clamp(0,1)
[0].permute(1,2,0).numpy()*255).astype('uint8')
    import easyocr
    reader = easyocr.Reader(['en'], gpu=False, model_storage_directory=str(CACHE/'easyocr'),
                            user_network_directory=str(CACHE/'easyocr/user'))
```

```
    def distance(a,b):
        previous = list(range(len(b)+1))
        for i,ca in enumerate(a,1):
            current = [i]
            for j,cb in enumerate(b,1):
                current.append(min(current[-1]+1,previous[j]+1,previous[j-1]+(ca!=cb)))
            previous=current
        return previous[-1]
    rows=[]
    for name, im in [('original',np.array(label)),('bicubic',cubic),('Real-ESRGAN',neural_text)]:
        file=OUT/f'ocr_{name}.png'
        Image.fromarray(im).save(file)
        tess=subprocess.run(['tesseract',str(file),'stdout','-l','eng','--psm','7'],
                            capture_output=True,text=True,check=True).stdout.strip()
        easy=' '.join(reader.readtext(im,detail=0))
        rows.append({'variant':name,'reference':reference_text,'Tesseract':tess,'EasyOCR':easy,
                     'CER_tesseract':distance(reference_text,tess)/len(reference_text),
                     'CER_easyocr':distance(reference_text,easy)/len(reference_text)})
    panel('ocr_comparison.png',[np.array(label),cubic,neural_text],
          ['Контрольная надпись','Уменьшение ×4 → Bicubic','Уменьшение ×4 → Real-ESRGAN'],cols=1)
    save('ocr_actual.json',rows)
    print('Completed human, SR and OCR controls', flush=True)

if __name__ == '__main__':
    main()
```

Листинг — Архитектура сети RRDBNet. Полный исходный текст файла RRDBNet_arch.py.

```
import functools
import torch
import torch.nn as nn
import torch.nn.functional as F


def make_layer(block, n_layers):
    layers = []
    for _ in range(n_layers):
        layers.append(block())
    return nn.Sequential(*layers)


class ResidualDenseBlock_5C(nn.Module):
    def __init__(self, nf=64, gc=32, bias=True):
        super(ResidualDenseBlock_5C, self).__init__()
        # gc: growth channel, i.e. intermediate channels
        self.conv1 = nn.Conv2d(nf, gc, 3, 1, 1, bias=bias)
        self.conv2 = nn.Conv2d(nf + gc, gc, 3, 1, 1, bias=bias)
        self.conv3 = nn.Conv2d(nf + 2 * gc, gc, 3, 1, 1, bias=bias)
        self.conv4 = nn.Conv2d(nf + 3 * gc, gc, 3, 1, 1, bias=bias)
        self.conv5 = nn.Conv2d(nf + 4 * gc, nf, 3, 1, 1, bias=bias)
        self.lrelu = nn.LeakyReLU(negative_slope=0.2, inplace=True)

        # initialization
        # mutil.initialize_weights([self.conv1, self.conv2, self.conv3, self.conv4, self.conv5],
0.1)

    def forward(self, x):
        x1 = self.lrelu(self.conv1(x))
        x2 = self.lrelu(self.conv2(torch.cat((x, x1), 1)))
        x3 = self.lrelu(self.conv3(torch.cat((x, x1, x2), 1)))
        x4 = self.lrelu(self.conv4(torch.cat((x, x1, x2, x3), 1)))
        x5 = self.conv5(torch.cat((x, x1, x2, x3, x4), 1))
        return x5 * 0.2 + x


class RRDB(nn.Module):
    '''Residual in Residual Dense Block'''

    def __init__(self, nf, gc=32):
        super(RRDB, self).__init__()
        self.RDB1 = ResidualDenseBlock_5C(nf, gc)
        self.RDB2 = ResidualDenseBlock_5C(nf, gc)
        self.RDB3 = ResidualDenseBlock_5C(nf, gc)
```

```
    def forward(self, x):
        out = self.RDB1(x)
        out = self.RDB2(out)
        out = self.RDB3(out)
        return out * 0.2 + x


class RRDBNet(nn.Module):
    def __init__(self, in_nc, out_nc, nf, nb, gc=32):
        super(RRDBNet, self).__init__()
        RRDB_block_f = functools.partial(RRDB, nf=nf, gc=gc)

        self.conv_first = nn.Conv2d(in_nc, nf, 3, 1, 1, bias=True)
        self.RRDB_trunk = make_layer(RRDB_block_f, nb)
        self.trunk_conv = nn.Conv2d(nf, nf, 3, 1, 1, bias=True)
        #### upsampling
        self.upconv1 = nn.Conv2d(nf, nf, 3, 1, 1, bias=True)
        self.upconv2 = nn.Conv2d(nf, nf, 3, 1, 1, bias=True)
        self.HRconv = nn.Conv2d(nf, nf, 3, 1, 1, bias=True)
        self.conv_last = nn.Conv2d(nf, out_nc, 3, 1, 1, bias=True)

        self.lrelu = nn.LeakyReLU(negative_slope=0.2, inplace=True)

    def forward(self, x):
        fea = self.conv_first(x)
        trunk = self.trunk_conv(self.RRDB_trunk(fea))
        fea = fea + trunk

        fea = self.lrelu(self.upconv1(F.interpolate(fea, scale_factor=2, mode='nearest')))
        fea = self.lrelu(self.upconv2(F.interpolate(fea, scale_factor=2, mode='nearest')))
        out = self.conv_last(self.lrelu(self.HRconv(fea)))

        return out
```

4. Поиск похожих изображений

4.1. Подготовка базы и представление признаков

База содержит 80 изображений 224 × 224: из каждого исходника формируется полный вид и четыре квадранта. Метки задают три группы: Treponema, палочковидные бактерии и кокки. CLIP переводит изображение и текст в общее пространство. ResNet-18 с удалённым классификатором выдаёт 512 признаков изображения. Векторы нормируются по L2, поэтому их скалярное произведение равно косинусному сходству.

Алгоритм выполнения. 1) Прочитать список исходных файлов и назначенных частей набора. 2) Сохранить фрагменты одинакового размера. 3) Вычислить эмбеддинги CLIP и ResNet-18 пакетами. 4) Нормировать векторы и записать NPY. 5) Получить вектор запроса. 6) Отсортировать базу по убыванию сходства. 7) Показать top-3 и оценить precision@3.

Таблица 12

Количество	**Значение**
Всего	80
Train	40
Validation	15
Test	25

4.2. Поиск по тексту и изображению

Таблица 13

Текст запроса	**Три ближайших файла и cosine**
Treponema pallidum spiral spirochetes	t02_0.jpg: 0.305060; t04_1.jpg: 0.301285; t04_3.jpg: 0.301239
rod shaped bacteria	s04_1.jpg: 0.325072; s01_4.jpg: 0.311313; s04_0.jpg: 0.311203
round cocci bacteria	t01_4.jpg: 0.293850; s02_0.jpg: 0.293142; t01_0.jpg: 0.287585

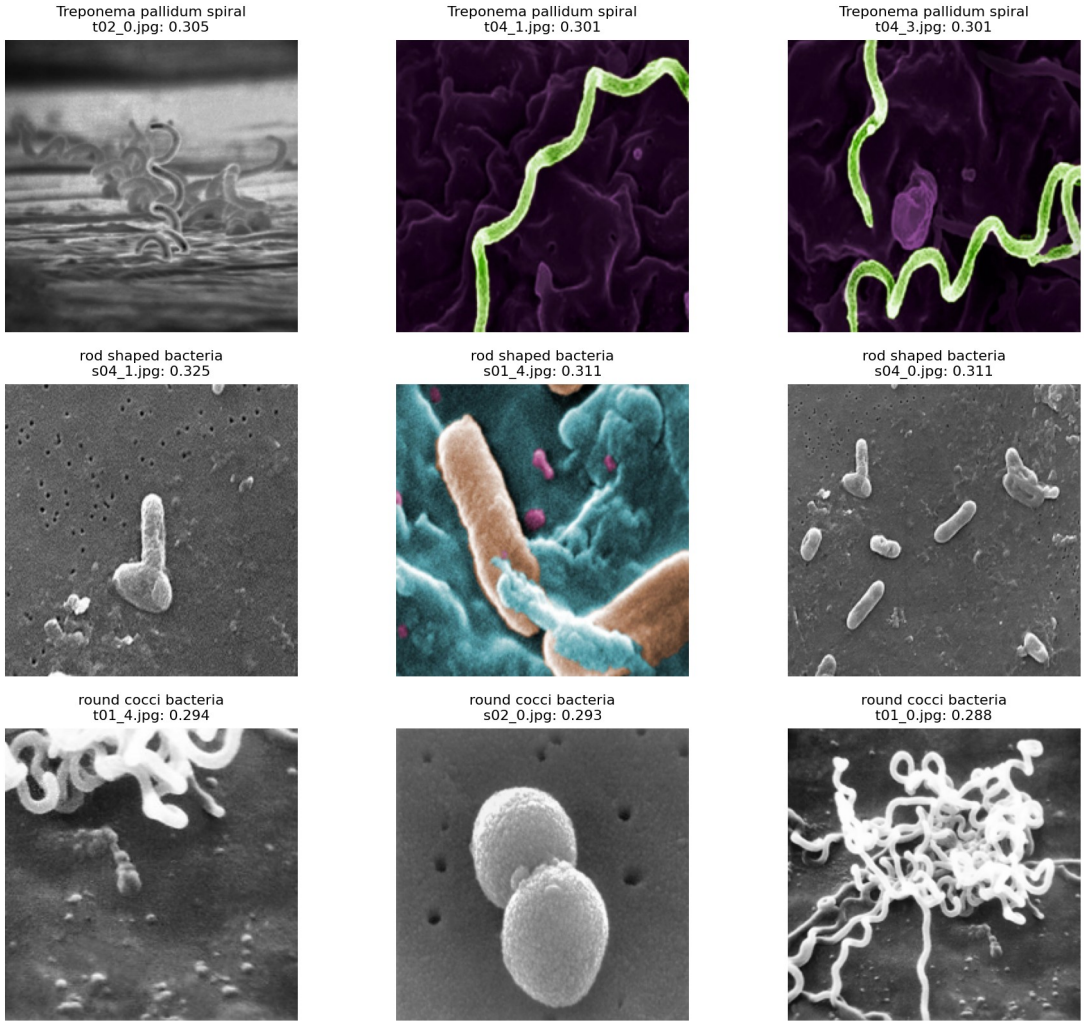


Рисунок 16 — Результаты CLIP для трёх текстовых запросов

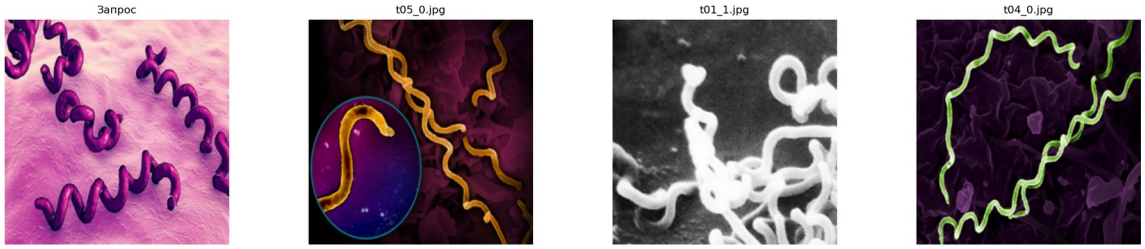


Рисунок 17 — Поиск ResNet-18 по изображению: запрос и три ближайших результата из других исходников

Для запроса Treponema pallidum spiral spirochetes первые результаты относятся к t02 и t04. Для rod shaped bacteria выдача содержит s04 и s01. Запрос round cocci bacteria также возвращает фрагменты трепонемы: модель сопоставляет визуальные и текстовые признаки и ранжирует доступную галерею, поэтому наиболее близкий результат не всегда совпадает с морфологическим классом.

Таблица 14

Модель	**Precision@3, исключён тот же источник**
CLIP	0.306667
ResNet-18	0.213333

Метрика вычислена для 25 тестовых запросов; кандидатами служат другие тестовые изображения, кроме фрагментов того же исходного файла. Для каждого запроса доля совпадений класса среди трёх ближайших усредняется. В тесте представлен один исходный файл кокков, поэтому после исключения того же источника для этого класса отсутствуют положительные кандидаты. Значения 0,3067 и 0,2133 описывают именно состав данной галереи и выбранное строгое исключение источника.

4.3. Полная программа

Программа run_learning.py выполняет единый воспроизводимый цикл: построение базы, поиск, обучение проекции и регистрацию классификаторов. Здесь и в разделах 5 и 9 приведён полный файл, чтобы листинги каждого пункта содержали и создание данных, и вычисление результатов. Входные файлы и их распределение явно перечислены в assignments.

Листинг — run_learning.py. Полный исходный текст файла run_learning.py.

```
"""CLIP/ResNet retrieval, triplet projection, 8 trees and 5 MLP runs in MLflow."""
from experiment_common import *
from PIL import Image
from torch import nn
from torch.nn import functional as F
from torchvision.models import resnet18, ResNet18_Weights
from transformers import CLIPModel, CLIPProcessor
from sklearn.metrics import accuracy_score, balanced_accuracy_score, f1_score, roc_auc_score,
confusion_matrix
from sklearn.tree import DecisionTreeClassifier
import mlflow
import mlflow.sklearn
import mlflow.pytorch
import itertools

def main():
    # Near-duplicate colour variants t04/t05 are kept together in train.
    assignments=[('t01.jpg',0,'train'),('t02.jpg',0,'train'),('t04.jpg',0,'train'),
                 ('t05.jpg',0,'train'),('t03.png',0,'val'),('t06.jpg',0,'test'),
                 ('treponema_primary.jpg',0,'test'),
                 ('s01.jpg',1,'train'),('s05.jpg',1,'train'),('s06.jpg',1,'train'),
                 ('s03.jpg',1,'val'),('s04.jpg',1,'test'),('s09.jpg',1,'test'),
                 ('s07.jpg',2,'train'),('s08.jpg',2,'val'),('s02.jpg',2,'test')]
    gallery=ROOT/'data/final_gallery';gallery.mkdir(parents=True,exist_ok=True)
    images=[];rows=[]
    for filename,label,split in assignments:
        path=(LEGACY if filename.startswith('s') else ROOT)/'data/raw'/filename
        image=Image.open(path).convert('RGB');w,h=image.size
```

```
        for j,box in enumerate([(0,0,w,h),(0,0,w//2,h//2),(w//2,0,w,h//2),
                                (0,h//2,w//2,h),(w//2,h//2,w,h)]):
            patch=image.crop(box).resize((224,224));name=Path(filename).stem+f'_{j}.jpg'
            patch.save(gallery/name);images.append(patch)
            rows.append({'file':name,'source':filename,'label':label,'split':split})
    save('gallery.json',rows)
    clip=CLIPModel.from_pretrained('openai/clip-vit-base-patch32',use_safetensors=True).eval()
    processor=CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
    clip_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):

clip_vectors.append(F.normalize(clip.get_image_features(**processor(images=images[i:i+8],
                                    return_tensors='pt')),dim=1).numpy())
    clip_vectors=np.concatenate(clip_vectors)

weights=ResNet18_Weights.DEFAULT;resnet=resnet18(weights=weights).eval();resnet.fc=nn.Identity()
    resnet_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):
            resnet_vectors.append(F.normalize(resnet(torch.stack([weights.transforms()(im)
                                for im in images[i:i+8]])),dim=1).numpy())
    resnet_vectors=np.concatenate(resnet_vectors)
    np.save(OUT/'clip_vectors.npy',clip_vectors);np.save(OUT/'resnet_vectors.npy',resnet_vectors)
    labels=np.array([r['label'] for r in rows]);splits=np.array([r['split'] for r in rows])

train=np.flatnonzero(splits=='train');val=np.flatnonzero(splits=='val');test=np.flatnonzero(split
s=='test')
    prompts=['Treponema pallidum spiral spirochetes','rod shaped bacteria','round cocci
bacteria']
    with
torch.no_grad():q=F.normalize(clip.get_text_features(**processor(text=prompts,padding=True,
                                                return_tensors='pt')),dim=1).numpy()
    hits=[];panels=[];titles=[]
    for prompt,vector in zip(prompts,q):
        scores=clip_vectors@vector;ids=np.argsort(-scores)[:3]
        hits.append({'query':prompt,'hits':[{'file':rows[i]['file'],'cosine':float(scores[i])}
for i in ids]})
        for i in ids:panels.append(np.array(images[i]));titles.append(f'{prompt[:25]}\n{rows[i]
["file"]}: {scores[i]:.3f}')
    panel('retrieval_text.png',panels,titles)
    def precision(vectors):
        values=[]
        for i in test:
            candidates=[j for j in test if rows[i]['source']!=rows[j]['source']]
            order=sorted(candidates,key=lambda j:float(vectors[i]@vectors[j]),reverse=True)[:3]
            values.append(float(np.mean(labels[order]==labels[i])))
        return float(np.mean(values))
    query_index=next(i for i,r in enumerate(rows) if r['file']=='treponema_primary_0.jpg')
    candidates=[i for i,r in enumerate(rows) if r['source']!='treponema_primary.jpg']
    idx=sorted(candidates,key=lambda
i:float(resnet_vectors[query_index]@resnet_vectors[i]),reverse=True)[:3]
    panel('retrieval_image.png',[np.array(images[query_index])]+[np.array(images[i]) for i in
idx],
          ['Запрос']+[rows[i]['file'] for i in idx],cols=4)
    save('retrieval_actual.json',
{'samples':len(rows),'train':len(train),'val':len(val),'test':len(test),
       'classes':['Treponema','rods','cocci'],'text_queries':hits,
       'CLIP_precision_at_3_cross_source':precision(clip_vectors),
       'ResNet_precision_at_3_cross_source':precision(resnet_vectors)})

    x=torch.tensor(resnet_vectors,dtype=torch.float32)
    projection=nn.Sequential(nn.Linear(512,128),nn.ReLU(),nn.Linear(128,64))
    optimizer=torch.optim.Adam(projection.parameters(),lr=.001)
    rng=np.random.default_rng(7);history=[];best=None;best_score=-1
    def pairs(ids):
        # Pairs share patches and may share a source; they are not independent samples.
        return list(itertools.combinations(ids,2))
    def evaluate(z,ids,threshold=None):
        pp=pairs(ids);truth=np.array([labels[i]==labels[j] for i,j in
pp]);scores=np.array([z[i]@z[j] for i,j in pp])
        if threshold is None:
            choices=np.linspace(-1,1,401)
            predictions=scores[:,None]>=choices
            balanced=(predictions[truth].mean(0)+(~predictions[~truth]).mean(0))/2
            threshold=choices[int(np.argmax(balanced))]
        return
{'threshold':float(threshold),'balanced_accuracy':float(balanced_accuracy_score(truth,scores>=thr
```

```
eshold)),
                'AUC':float(roc_auc_score(truth,scores)),'pairs':len(pp)}
    for epoch in range(100):
        triples=[]
        for i in train:
            positives=[j for j in train if labels[j]==labels[i] and j!=i]
            negatives=[j for j in train if labels[j]!=labels[i]]
            triples.append([i,rng.choice(positives),rng.choice(negatives)])
        t=np.array(triples);z=F.normalize(projection(x),dim=1)
        loss=F.triplet_margin_loss(z[t[:,0]],z[t[:,1]],z[t[:,2]],margin=.5)
        optimizer.zero_grad();loss.backward();optimizer.step()
        with torch.no_grad():vectors=F.normalize(projection(x),dim=1).numpy()
        validation=evaluate(vectors,val)
        if validation['balanced_accuracy']>best_score:
            best_score=validation['balanced_accuracy'];best={k:v.detach().clone() for k,v in
projection.state_dict().items()}
        history.append(float(loss.detach()))
    projection.load_state_dict(best)
    with torch.no_grad():learned=F.normalize(projection(x),dim=1).numpy()
    baseline=evaluate(resnet_vectors,val);after=evaluate(learned,val)
    metric={'base':evaluate(resnet_vectors,test,baseline['threshold']),
            'triplet':evaluate(learned,test,after['threshold']), 'train_loss':history}
    torch.save(projection.state_dict(),OUT/'triplet_head.pt');save('metric_actual.json',metric)

fig,ax=plt.subplots(figsize=(8,3));ax.plot(range(1,101),history);ax.set(xlabel='Эпоха',ylabel='Tr
iplet loss')
    fig.tight_layout();fig.savefig(OUT/'triplet_learning.png',dpi=160);plt.close(fig)

    mlflow.set_tracking_uri('sqlite:///'+str(CACHE/'mlflow.db'))
    mlflow.set_experiment('Treponema_LR4_measured')
    mlflow_runs=[];tree_models=[]
    for depth in [2,4,8,16]:
        for minimum in [2,10]:

estimator=DecisionTreeClassifier(max_depth=depth,min_samples_split=minimum,random_state=7).fit(re
snet_vectors[train],labels[train])

pred=estimator.predict(resnet_vectors[test]);vpred=estimator.predict(resnet_vectors[val])
            with mlflow.start_run(run_name=f'tree_{depth}_{minimum}') as run:
                mlflow.log_params({'depth':depth,'min_samples_split':minimum,'backbone':'ResNet18
frozen'})
                scores={'val_f1':float(f1_score(labels[val],vpred,average='macro')),
                        'test_f1':float(f1_score(labels[test],pred,average='macro'))}
                mlflow.log_metrics(scores)
                info=mlflow.sklearn.log_model(estimator,name='model',
                         skops_trusted_types=['sklearn.tree._tree.Tree'])
                mlflow_runs.append({'name':f'tree_{depth}
_{minimum}','run_id':run.info.run_id,**scores})
                tree_models.append((scores['val_f1'],info.model_uri,estimator))
    network_runs=[]
    for hidden,dropout in [(32,0),(64,0),(64,.2),(128,.2),(128,.5)]:
        torch.manual_seed(7)

net=nn.Sequential(nn.Linear(512,hidden),nn.ReLU(),nn.Dropout(dropout),nn.Linear(hidden,3))
        optim=torch.optim.Adam(net.parameters(),lr=.005);ys=torch.tensor(labels,dtype=torch.long)
        values=[]
        with mlflow.start_run(run_name=f'mlp_{hidden}_{dropout}') as run:
            mlflow.log_params({'hidden':hidden,'dropout':dropout,'epochs':60,'backbone':'ResNet18
frozen'})
            for epoch in range(60):

net.train();loss=F.cross_entropy(net(x[train]),ys[train]);optim.zero_grad();loss.backward();optim
.step()
                net.eval()
                with torch.no_grad():v=net(x[val]).argmax(1).numpy()
                score=float(accuracy_score(labels[val],v));values.append(score)
                mlflow.log_metric('val_accuracy',score,step=epoch)
            with torch.no_grad():pred=net(x[test]).argmax(1).numpy()

score=float(f1_score(labels[test],pred,average='macro'));mlflow.log_metric('test_f1',score)
            info=mlflow.pytorch.log_model(net,name='model',input_example=resnet_vectors[test])
            loaded=mlflow.pyfunc.load_model(info.model_uri)
            restored_prediction=loaded.predict(resnet_vectors[test])
            same=bool(np.array_equal(np.asarray(restored_prediction).argmax(1),pred))
            network_runs.append({'name':f'mlp_{hidden}_{dropout}','run_id':run.info.run_id,
                                 'test_f1':score,'val_history':values,'reload_identical':same})
    registered=[]
    for _,uri,estimator in sorted(tree_models,key=lambda t:t[0],reverse=True)[:2]:
```

```
        version=mlflow.register_model(uri,'Treponema_ResNet_Tree')
        restored=mlflow.sklearn.load_model(f'models:/Treponema_ResNet_Tree/{version.version}')

identical=bool(np.array_equal(restored.predict(resnet_vectors[test]),estimator.predict(resnet_vec
tors[test])))

registered.append({'name':version.name,'version':version.version,'reload_identical':identical})
    save('mlflow_actual.json',
{'trees':mlflow_runs,'networks':network_runs,'registered':registered})
    fig,axes=plt.subplots(1,2,figsize=(12,4))
    runs=mlflow_runs+network_runs;axes[0].bar(range(len(runs)),[r['test_f1'] for r in runs])
    axes[0].set_xticks(range(len(runs)),[r['name'] for r in
runs],rotation=70,fontsize=7);axes[0].set_ylabel('Test macro-F1')
    for r in network_runs:axes[1].plot(r['val_history'],label=r['name'])
    axes[1].set(xlabel='Эпоха',ylabel='Validation accuracy');axes[1].legend(fontsize=7)
    fig.tight_layout();fig.savefig(OUT/'mlflow_actual.png',dpi=160);plt.close(fig)
    print('Completed retrieval, metric learning and 13 MLflow runs',flush=True)

if __name__=='__main__':
    main()
```

5. Обучение метрики сходства

5.1. Тройки и функция потерь

Обучение метрики изменяет пространство признаков так, чтобы объекты одной группы находились ближе друг к другу. Тройка состоит из anchor, positive того же класса и negative другого класса. Используется функция $L = \max(0, \|a - p\|_2 - \|a - n\|_2 + \text{margin})$. В опыте margin = 0,5. L2-нормировка ограничивает масштаб представления и препятствует уменьшению потерь простым увеличением норм векторов.

Вход — сохранённые 512-мерные признаки ResNet-18 и разбиение 40/15/25. Обучается проекция Linear(512,128) → ReLU → Linear(128,64), после которой выполняется нормировка. Backbone ResNet-18 фиксирован. Adam использует learning rate 0,001; обучение продолжается 100 эпох. Для каждого обучающего изображения выбирается положительный пример с другой позицией в базе и отрицательный пример другого класса.

Алгоритм выполнения. 1) Получить базовые эмбеддинги. 2) Сформировать тройки только из train. 3) Выполнить прямой проход проекции и нормировку. 4) Вычислить triplet loss и обновить веса. 5) На validation подобрать порог и сохранить состояние с лучшей balanced accuracy. 6) Применить выбранный порог к 300 парам test. 7) Сравнить результат с исходными признаками.

5.2. Потери, калибровка порога и результаты

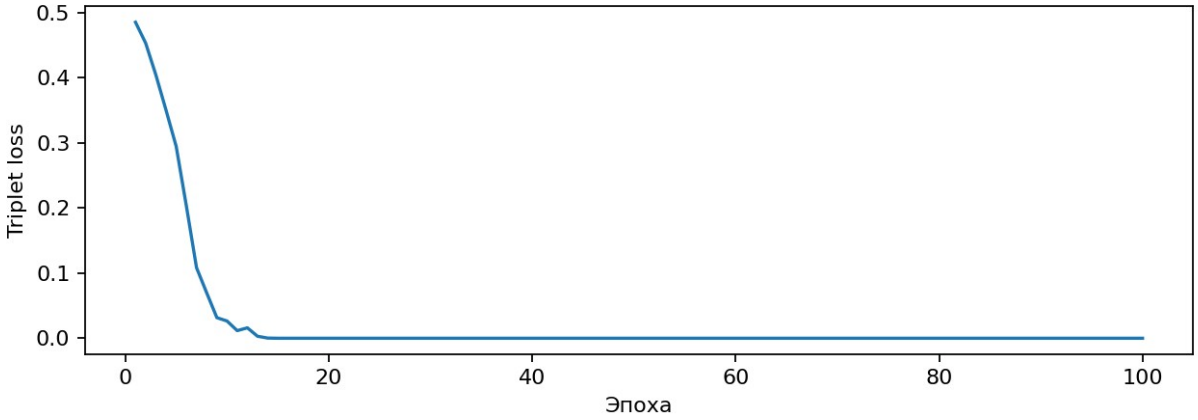


Рисунок 18 — Изменение triplet loss за 100 эпох обучения проекции

Таблица 15

Представление	**Порог cosine**	**Balanced accuracy**	**ROC AUC**	**Пар**
ResNet-18	0.590	0.7050	0.7947	300
Triplet projection	0.990	0.6925	0.7500	300

Начальная потеря равна 0,485057; к 15-й эпохе на сформированных обучающих тройках она достигает нуля. На тестовых парах базовые признаки дают balanced accuracy 0,7050 и ROC AUC 0,7947, проекция — 0,6925 и 0,7500. Таким образом, уменьшение обучающей функции не сопровождается увеличением качества на этих тестовых парах. Выбор модели основывается на проверочных измерениях, а не только на кривой train loss.

Порог выбирается по 401 значению от −1 до 1 на validation и затем фиксируется для test. Пары строятся из всех сочетаний тестовых фрагментов: C(25, 2) = 300. Они имеют общие изображения и могут происходить из одного исходника, поэтому 300 пар не эквивалентны 300 независимым микрофотографиям. В данном опыте классы сохраняются во всех частях набора; проверяется перенос между источниками внутри выбранных морфологических групп.

Выходные данные: triplet_head.pt с параметрами проекции, metric_actual.json с порогами и измерениями, triplet_learning.png с кривой потерь. Базовый и обученный варианты оцениваются одним правилом

построения пар. Для перехода к новым видам бактерий требуется отдельное разбиение по классам и достаточное число независимых источников каждого вида.

5.3. Полный листинг обучения и оценки

Листинг — run_learning.py. Полный исходный текст файла run_learning.py.

```
"""CLIP/ResNet retrieval, triplet projection, 8 trees and 5 MLP runs in MLflow."""
from experiment_common import *
from PIL import Image
from torch import nn
from torch.nn import functional as F
from torchvision.models import resnet18, ResNet18_Weights
from transformers import CLIPModel, CLIPProcessor
from sklearn.metrics import accuracy_score, balanced_accuracy_score, f1_score, roc_auc_score,
confusion_matrix
from sklearn.tree import DecisionTreeClassifier
import mlflow
import mlflow.sklearn
import mlflow.pytorch
import itertools

def main():
    # Near-duplicate colour variants t04/t05 are kept together in train.
    assignments=[('t01.jpg',0,'train'),('t02.jpg',0,'train'),('t04.jpg',0,'train'),
                 ('t05.jpg',0,'train'),('t03.png',0,'val'),('t06.jpg',0,'test'),
                 ('treponema_primary.jpg',0,'test'),
                 ('s01.jpg',1,'train'),('s05.jpg',1,'train'),('s06.jpg',1,'train'),
                 ('s03.jpg',1,'val'),('s04.jpg',1,'test'),('s09.jpg',1,'test'),
                 ('s07.jpg',2,'train'),('s08.jpg',2,'val'),('s02.jpg',2,'test')]
    gallery=ROOT/'data/final_gallery';gallery.mkdir(parents=True,exist_ok=True)
    images=[];rows=[]
    for filename,label,split in assignments:
        path=(LEGACY if filename.startswith('s') else ROOT)/'data/raw'/filename
        image=Image.open(path).convert('RGB');w,h=image.size
        for j,box in enumerate([(0,0,w,h),(0,0,w//2,h//2),(w//2,0,w,h//2),
                                (0,h//2,w//2,h),(w//2,h//2,w,h)]):
            patch=image.crop(box).resize((224,224));name=Path(filename).stem+f'_{j}.jpg'
            patch.save(gallery/name);images.append(patch)
            rows.append({'file':name,'source':filename,'label':label,'split':split})
    save('gallery.json',rows)
    clip=CLIPModel.from_pretrained('openai/clip-vit-base-patch32',use_safetensors=True).eval()
    processor=CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
    clip_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):

clip_vectors.append(F.normalize(clip.get_image_features(**processor(images=images[i:i+8],
                                    return_tensors='pt')),dim=1).numpy())
    clip_vectors=np.concatenate(clip_vectors)

weights=ResNet18_Weights.DEFAULT;resnet=resnet18(weights=weights).eval();resnet.fc=nn.Identity()
    resnet_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):
            resnet_vectors.append(F.normalize(resnet(torch.stack([weights.transforms()(im)
                                for im in images[i:i+8]])),dim=1).numpy())
    resnet_vectors=np.concatenate(resnet_vectors)
    np.save(OUT/'clip_vectors.npy',clip_vectors);np.save(OUT/'resnet_vectors.npy',resnet_vectors)
    labels=np.array([r['label'] for r in rows]);splits=np.array([r['split'] for r in rows])

train=np.flatnonzero(splits=='train');val=np.flatnonzero(splits=='val');test=np.flatnonzero(split
s=='test')
    prompts=['Treponema pallidum spiral spirochetes','rod shaped bacteria','round cocci
bacteria']
    with
torch.no_grad():q=F.normalize(clip.get_text_features(**processor(text=prompts,padding=True,
                                            return_tensors='pt')),dim=1).numpy()
```

```
    hits=[];panels=[];titles=[]
    for prompt,vector in zip(prompts,q):
        scores=clip_vectors@vector;ids=np.argsort(-scores)[:3]
        hits.append({'query':prompt,'hits':[{'file':rows[i]['file'],'cosine':float(scores[i])}
for i in ids]})
        for i in ids:panels.append(np.array(images[i]));titles.append(f'{prompt[:25]}\n{rows[i]
["file"]}: {scores[i]:.3f}')
    panel('retrieval_text.png',panels,titles)
    def precision(vectors):
        values=[]
        for i in test:
            candidates=[j for j in test if rows[i]['source']!=rows[j]['source']]
            order=sorted(candidates,key=lambda j:float(vectors[i]@vectors[j]),reverse=True)[:3]
            values.append(float(np.mean(labels[order]==labels[i])))
        return float(np.mean(values))
    query_index=next(i for i,r in enumerate(rows) if r['file']=='treponema_primary_0.jpg')
    candidates=[i for i,r in enumerate(rows) if r['source']!='treponema_primary.jpg']
    idx=sorted(candidates,key=lambda
i:float(resnet_vectors[query_index]@resnet_vectors[i]),reverse=True)[:3]
    panel('retrieval_image.png',[np.array(images[query_index])]+[np.array(images[i]) for i in
idx],
          ['Запрос']+[rows[i]['file'] for i in idx],cols=4)
    save('retrieval_actual.json',
{'samples':len(rows),'train':len(train),'val':len(val),'test':len(test),
       'classes':['Treponema','rods','cocci'],'text_queries':hits,
       'CLIP_precision_at_3_cross_source':precision(clip_vectors),
       'ResNet_precision_at_3_cross_source':precision(resnet_vectors)})

    x=torch.tensor(resnet_vectors,dtype=torch.float32)
    projection=nn.Sequential(nn.Linear(512,128),nn.ReLU(),nn.Linear(128,64))
    optimizer=torch.optim.Adam(projection.parameters(),lr=.001)
    rng=np.random.default_rng(7);history=[];best=None;best_score=-1
    def pairs(ids):
        # Pairs share patches and may share a source; they are not independent samples.
        return list(itertools.combinations(ids,2))
    def evaluate(z,ids,threshold=None):
        pp=pairs(ids);truth=np.array([labels[i]==labels[j] for i,j in
pp]);scores=np.array([z[i]@z[j] for i,j in pp])
        if threshold is None:
            choices=np.linspace(-1,1,401)
            predictions=scores[:,None]>=choices
            balanced=(predictions[truth].mean(0)+(~predictions[~truth]).mean(0))/2
            threshold=choices[int(np.argmax(balanced))]
        return
{'threshold':float(threshold),'balanced_accuracy':float(balanced_accuracy_score(truth,scores>=thr
eshold)),
                'AUC':float(roc_auc_score(truth,scores)),'pairs':len(pp)}
    for epoch in range(100):
        triples=[]
        for i in train:
            positives=[j for j in train if labels[j]==labels[i] and j!=i]
            negatives=[j for j in train if labels[j]!=labels[i]]
            triples.append([i,rng.choice(positives),rng.choice(negatives)])
        t=np.array(triples);z=F.normalize(projection(x),dim=1)
        loss=F.triplet_margin_loss(z[t[:,0]],z[t[:,1]],z[t[:,2]],margin=.5)
        optimizer.zero_grad();loss.backward();optimizer.step()
        with torch.no_grad():vectors=F.normalize(projection(x),dim=1).numpy()
        validation=evaluate(vectors,val)
        if validation['balanced_accuracy']>best_score:
            best_score=validation['balanced_accuracy'];best={k:v.detach().clone() for k,v in
projection.state_dict().items()}
        history.append(float(loss.detach()))
    projection.load_state_dict(best)
    with torch.no_grad():learned=F.normalize(projection(x),dim=1).numpy()
    baseline=evaluate(resnet_vectors,val);after=evaluate(learned,val)
    metric={'base':evaluate(resnet_vectors,test,baseline['threshold']),
            'triplet':evaluate(learned,test,after['threshold']), 'train_loss':history}
    torch.save(projection.state_dict(),OUT/'triplet_head.pt');save('metric_actual.json',metric)

fig,ax=plt.subplots(figsize=(8,3));ax.plot(range(1,101),history);ax.set(xlabel='Эпоха',ylabel='Tr
iplet loss')
    fig.tight_layout();fig.savefig(OUT/'triplet_learning.png',dpi=160);plt.close(fig)

    mlflow.set_tracking_uri('sqlite:///'+str(CACHE/'mlflow.db'))
    mlflow.set_experiment('Treponema_LR4_measured')
    mlflow_runs=[];tree_models=[]
    for depth in [2,4,8,16]:
        for minimum in [2,10]:
```

```
estimator=DecisionTreeClassifier(max_depth=depth,min_samples_split=minimum,random_state=7).fit(re
snet_vectors[train],labels[train])

pred=estimator.predict(resnet_vectors[test]);vpred=estimator.predict(resnet_vectors[val])
            with mlflow.start_run(run_name=f'tree_{depth}_{minimum}') as run:
                mlflow.log_params({'depth':depth,'min_samples_split':minimum,'backbone':'ResNet18
frozen'})
                scores={'val_f1':float(f1_score(labels[val],vpred,average='macro')),
                        'test_f1':float(f1_score(labels[test],pred,average='macro'))}
                mlflow.log_metrics(scores)
                info=mlflow.sklearn.log_model(estimator,name='model',
                         skops_trusted_types=['sklearn.tree._tree.Tree'])
                mlflow_runs.append({'name':f'tree_{depth}
_{minimum}','run_id':run.info.run_id,**scores})
                tree_models.append((scores['val_f1'],info.model_uri,estimator))
    network_runs=[]
    for hidden,dropout in [(32,0),(64,0),(64,.2),(128,.2),(128,.5)]:
        torch.manual_seed(7)

net=nn.Sequential(nn.Linear(512,hidden),nn.ReLU(),nn.Dropout(dropout),nn.Linear(hidden,3))
        optim=torch.optim.Adam(net.parameters(),lr=.005);ys=torch.tensor(labels,dtype=torch.long)
        values=[]
        with mlflow.start_run(run_name=f'mlp_{hidden}_{dropout}') as run:
            mlflow.log_params({'hidden':hidden,'dropout':dropout,'epochs':60,'backbone':'ResNet18
frozen'})
            for epoch in range(60):

net.train();loss=F.cross_entropy(net(x[train]),ys[train]);optim.zero_grad();loss.backward();optim
.step()
                net.eval()
                with torch.no_grad():v=net(x[val]).argmax(1).numpy()
                score=float(accuracy_score(labels[val],v));values.append(score)
                mlflow.log_metric('val_accuracy',score,step=epoch)
            with torch.no_grad():pred=net(x[test]).argmax(1).numpy()

score=float(f1_score(labels[test],pred,average='macro'));mlflow.log_metric('test_f1',score)
            info=mlflow.pytorch.log_model(net,name='model',input_example=resnet_vectors[test])
            loaded=mlflow.pyfunc.load_model(info.model_uri)
            restored_prediction=loaded.predict(resnet_vectors[test])
            same=bool(np.array_equal(np.asarray(restored_prediction).argmax(1),pred))
            network_runs.append({'name':f'mlp_{hidden}_{dropout}','run_id':run.info.run_id,
                                 'test_f1':score,'val_history':values,'reload_identical':same})
    registered=[]
    for _,uri,estimator in sorted(tree_models,key=lambda t:t[0],reverse=True)[:2]:
        version=mlflow.register_model(uri,'Treponema_ResNet_Tree')
        restored=mlflow.sklearn.load_model(f'models:/Treponema_ResNet_Tree/{version.version}')

identical=bool(np.array_equal(restored.predict(resnet_vectors[test]),estimator.predict(resnet_vec
tors[test])))

registered.append({'name':version.name,'version':version.version,'reload_identical':identical})
    save('mlflow_actual.json',
{'trees':mlflow_runs,'networks':network_runs,'registered':registered})
    fig,axes=plt.subplots(1,2,figsize=(12,4))
    runs=mlflow_runs+network_runs;axes[0].bar(range(len(runs)),[r['test_f1'] for r in runs])
    axes[0].set_xticks(range(len(runs)),[r['name'] for r in
runs],rotation=70,fontsize=7);axes[0].set_ylabel('Test macro-F1')
    for r in network_runs:axes[1].plot(r['val_history'],label=r['name'])
    axes[1].set(xlabel='Эпоха',ylabel='Validation accuracy');axes[1].legend(fontsize=7)
    fig.tight_layout();fig.savefig(OUT/'mlflow_actual.png',dpi=160);plt.close(fig)
    print('Completed retrieval, metric learning and 13 MLflow runs',flush=True)

if __name__=='__main__':
    main()
```

6. Нейросети без обучения: zero-shot

6.1. Модели и входные запросы

Входом всех опытов служит одно основное изображение трепонемы. YOLO-World выполняет детекцию по словарю классов; FastSAM формирует маски и выбирает объект по тексту; CLIP сравнивает изображение с описаниями; CLIPSeg строит карту соответствия запросу; Grounding DINO выдаёт рамки, которые передаются в SAM. Эти выходы имеют разный смысл, поэтому число рамок, число масок и cosine приводятся отдельно.

Алгоритм выполнения. 1) Загрузить готовые веса. 2) Задать словарь и текстовые запросы. 3) Получить рамки YOLO-World при conf = 0,1. 4) Получить FastSAM-маску по запросу a spiral bacterium. 5) Сравнить четыре описания CLIP. 6) Получить карты CLIPSeg для общей формы и видового названия. 7) Передать рамки Grounding DINO при пороге 0,2 в SAM. 8) Сохранить изображения и численные выходы.

Таблица 16

Метод	**Выход на основном изображении**
YOLO-World	Рамок: 0; conf = 0,1
FastSAM + текст	Масок: 1
CLIPSeg: spiral bacterium	Доля маски: 0.283938
CLIPSeg: Treponema pallidum	Доля маски: 0.000000
Grounding DINO	Рамок: 7; порог = 0,2
SAM по рамкам DINO	Масок: 7; доля объединения: 0.933525

Таблица 17

Описание CLIP	**Косинусное сходство**
a spiral spirochete bacterium	0.288219
a rod shaped bacterium	0.277365
round cocci bacteria	0.266132
a human person	0.193371

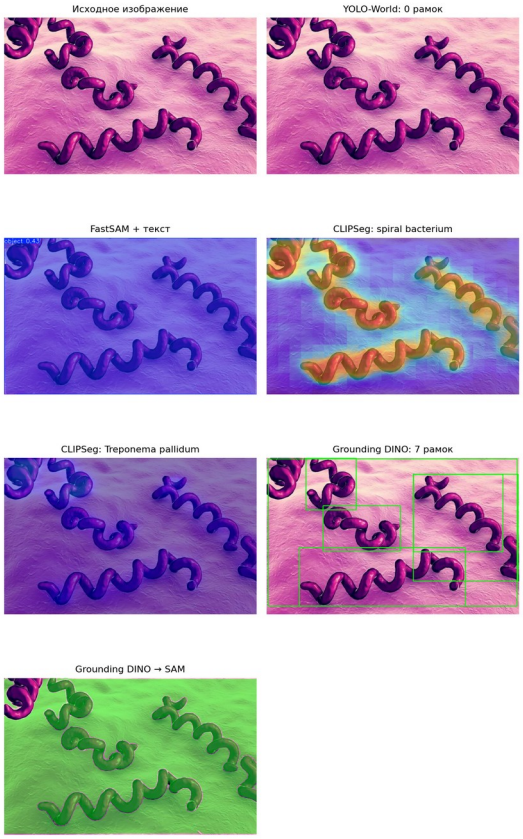


Рисунок 19 — Выходы YOLO-World, FastSAM, CLIPSeg и Grounding DINO + SAM на изображении трепонемы

6.2. Анализ полученных выходов

CLIP присваивает наибольшее сходство описанию a spiral spirochete bacterium: 0,288219. Описание человека имеет 0,193371. Это относительное ранжирование заданных описаний. Даже максимальное значение не является вероятностью медицинского диагноза. YOLO-World при выбранном словаре и пороге возвращает ноль рамок, FastSAM выбирает одну маску.

Общий запрос spiral bacterium у CLIPSeg выделяет 28,39 % площади, а точное видовое название — 0 %. Формулировка текста заметно влияет на результат. Grounding DINO возвращает семь рамок с оценками от 0,2041 до 0,5283; часть рамок пересекается, одна охватывает почти весь кадр. Объединение семи SAM-масок занимает 93,35 % площади. При такой подсказке большая маска включает фон: качество итоговой сегментации зависит от качества исходных рамок.

Для дальнейшей разметки пригодность маски оценивается по её границам и охвату нужного фрагмента. Цветовая маска основной иллюстрации служит геометрическим ориентиром для сопоставления; она основана на другом признаке и не превращает оценки универсальных моделей в метрики видовой идентификации.

6.3. Полная программа

Листинг — run_zero_shot.py. Полный исходный текст файла run_zero_shot.py.

```
"""Measure five zero-shot approaches on the supplied Treponema illustration."""
from experiment_common import *
from PIL import Image
from ultralytics import YOLOWorld, FastSAM, settings
from transformers import (CLIPModel, CLIPProcessor, CLIPSegProcessor,
    CLIPSegForImageSegmentation, AutoProcessor, AutoModelForZeroShotObjectDetection,
    SamModel, SamProcessor)
from torch.nn import functional as F

def main():
    settings.update({'weights_dir':str(CACHE)})
    image=Image.open(ROOT/'data/raw/treponema_primary.jpg').convert('RGB')
    metrics={};figures=[np.array(image)];titles=['Исходное изображение']
    world=YOLOWorld(CACHE/'yolov8s-worldv2.pt')
    world.set_classes(['bacterium','spirochete','Treponema pallidum'])
    result=world(image,conf=.1,imgsz=640,device='cpu',verbose=False)[0]
    metrics['YOLO-World']={'threshold':.1,'boxes':len(result.boxes),
                           'confidence':result.boxes.conf.cpu().tolist()}
    figures.append(cv2.cvtColor(result.plot(),cv2.COLOR_BGR2RGB));titles.append(f'YOLO-World:
{len(result.boxes)} рамок')
    save('zero_actual.json',metrics)
    fast=FastSAM(CACHE/'FastSAM-s.pt')
    result=fast(image,texts='a spiral bacterium',conf=.25,imgsz=640,device='cpu',verbose=False)
[0]
    metrics['FastSAM']={'text':'a spiral bacterium','masks':0 if result.masks is None else
len(result.masks)}
    figures.append(cv2.cvtColor(result.plot(),cv2.COLOR_BGR2RGB));titles.append('FastSAM +
текст')
    save('zero_actual.json',metrics)
    clip=CLIPModel.from_pretrained('openai/clip-vit-base-patch32',use_safetensors=True).eval()
    processor=CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
    prompts=['a spiral spirochete bacterium','a rod shaped bacterium','round cocci bacteria','a
human person']
    with torch.no_grad():

a=F.normalize(clip.get_image_features(**processor(images=image,return_tensors='pt')),dim=1)

b=F.normalize(clip.get_text_features(**processor(text=prompts,padding=True,return_tensors='pt')),
dim=1)
    metrics['CLIP']=[{'prompt':p,'cosine':float(s)} for p,s in zip(prompts,(a@b.T)[0])]
    processor=CLIPSegProcessor.from_pretrained('CIDAS/clipseg-rd64-refined')
    net=CLIPSegForImageSegmentation.from_pretrained('CIDAS/clipseg-rd64-
refined',use_safetensors=True).eval()
    texts=['spiral bacterium','Treponema pallidum']
    with
torch.no_grad():maps=net(**processor(text=texts,images=[image]*2,padding=True,return_tensors='pt'
)).logits.sigmoid().numpy()
    metrics['CLIPSeg']=[]
    for text,prob in zip(texts,maps):
        prob=cv2.resize(prob,image.size)
        metrics['CLIPSeg'].append({'prompt':text,'mask_fraction':float((prob>.5).mean())})
        heat=cv2.applyColorMap((prob*255).astype('uint8'),cv2.COLORMAP_JET)
        overlay=cv2.addWeighted(np.array(image),.55,cv2.cvtColor(heat,cv2.COLOR_BGR2RGB),.45,0)
        figures.append(overlay);titles.append('CLIPSeg: '+text)
    save('zero_actual.json',metrics)
```

```
    processor=AutoProcessor.from_pretrained('IDEA-Research/grounding-dino-tiny')
    net=AutoModelForZeroShotObjectDetection.from_pretrained('IDEA-Research/grounding-dino-
tiny',use_safetensors=True).eval()
    inputs=processor(images=image,text='a spiral bacterium. a spirochete.',return_tensors='pt')
    with torch.no_grad():outputs=net(**inputs)
    result=processor.post_process_grounded_object_detection(outputs,inputs.input_ids,
             box_threshold=.20,text_threshold=.20,target_sizes=[image.size[::-1]])[0]
    boxes=result['boxes'].tolist()
    metrics['Grounding DINO']={'threshold':.20,'boxes':boxes,'scores':result['scores'].tolist()}
    shown=np.array(image)
    for box in boxes:
        x1,y1,x2,y2=map(int,box);cv2.rectangle(shown,(x1,y1),(x2,y2),(0,255,0),2)
    figures.append(shown);titles.append(f'Grounding DINO: {len(boxes)} рамок')
    if boxes:
        processor=SamProcessor.from_pretrained('facebook/sam-vit-base')
        net=SamModel.from_pretrained('facebook/sam-vit-base',use_safetensors=True).eval()
        inputs=processor(image,input_boxes=[boxes],return_tensors='pt')
        with torch.no_grad():outputs=net(**inputs,multimask_output=False)

masks=processor.image_processor.post_process_masks(outputs.pred_masks,inputs['original_sizes'],
                                                   inputs['reshaped_input_sizes'])[0][:,0].numpy()
        shown=np.array(image)
        union=masks.any(axis=0)
        shown[union]=(shown[union]*.5+np.array([0,255,0])*.5).astype('uint8')
        figures.append(shown);titles.append('Grounding DINO → SAM')
        metrics['SAM']={'masks':len(masks),'foreground_fraction':float(union.mean())}
    else:
        metrics['SAM']={'masks':0,'input_boxes':0}
    panel('zero_actual.png',figures,titles,cols=2)
    save('zero_actual.json',metrics)
    print(metrics,flush=True)

if __name__=='__main__':
    main()
```

7. Поиск момента в видео RuTube и интерфейс Streamlit

7.1. Входные данные и построение индекса

Для поиска выбран ролик «Наглядная микробиология #5 Бактерии (Bacteria)». Ссылка на исходное видео: https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/. Локальный запуск сформировал индекс из 81 кадра в каталоге data/rutube/index. В отчёте приведены сохранённые изображения интерфейса этого запуска.

Видео рассматривается как последовательность визуальных документов. С шагом 2 секунды выбирается кадр, сохраняется его время и вычисляется CLIP-эмбеддинг. Матрица векторов записывается в vectors.npy, сведения о кадрах — в index.json. При запросе обрабатывается локальный индекс: повторное скачивание ролика для поиска не требуется.

Алгоритм выполнения. 1) Получить локальный видеофайл и сохранить URL источника. 2) Читать кадры последовательно с помощью VideoCapture. 3) Сохранять кадры с заданным временным шагом. 4) Вычислить и нормировать CLIP-векторы. 5) Закодировать текстовый запрос или загруженное изображение. 6) Вычислить cosine и отсортировать результаты. 7) Отфильтровать близкие моменты и почти одинаковые векторы. 8) Показать кадр, время, сходство и ссылку с параметром ?t=.

7.2. Полученные результаты поиска

Введённый запрос: Treponema pallidum spirochetes moving in a microscope field. Скриншоты фиксируют состояние формы поиска и три карточки результата. Числа в таблице соответствуют значениям, отображённым на этих карточках; время относится к исходному ролику.

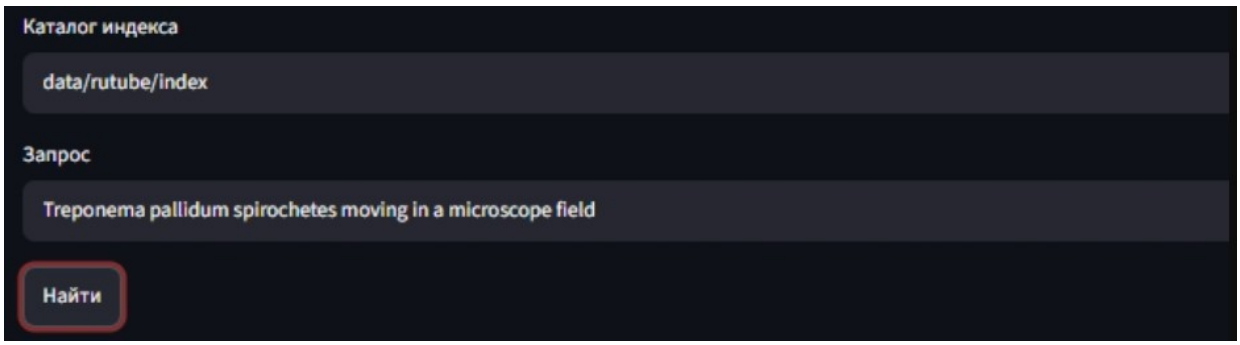


Рисунок 20 — Форма Streamlit с локальным каталогом индекса и текстовым запросом



Рисунок 21 — Три найденных момента ролика RuTube: кадры, время и сходство

Таблица 18

Место	**Время, с**	**Cosine**	**Ссылка на момент**
1	144	0.341	https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/?t=144
2	2	0.339	https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/?t=2
3	148	0.337	https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/?t=148

Первый и третий результаты показывают визуально близкие изображения палочковидных бактерий, второй — микроскопическое поле с подписью «Бактерии». Это результат смыслового поиска в общебиологическом ролике по тексту запроса. Сходство с запросом не меняет видимое содержание кадров: карточки являются найденными кандидатами для просмотра, а не подтверждением вида Treponema pallidum.

В сохранённой выдаче присутствуют моменты 144 и 148 секунд с похожими изображениями. Для последующих запросов интерфейс содержит настройку минимального временного интервала и дополнительно исключает пары векторов со сходством выше 0,98. Эти параметры позволяют регулировать разнообразие выдачи. Скриншоты сохраняют фактические результаты исходного запуска.

7.3. Запуск приложения и устройство файлов

В PowerShell команды выполняются из каталога lab4_treponema. Надёжный способ запуска — python -m streamlit run scripts/app.py: модуль Streamlit запускается через выбранный интерпретатор Python. Если команда python ссылается на другое окружение, используется полный путь к python.exe нужного виртуального окружения.

```
python -m streamlit run scripts/app.py
# В интерфейсе: каталог индекса data/rutube/index
# Запрос: Treponema pallidum spirochetes moving in a microscope field
```

Для использования готового индекса достаточно файлов index.json, vectors.npy и папки кадров. Поле каталога в интерфейсе задаёт их расположение. Вкладка загрузки изображения позволяет использовать визуальный запрос. При наличии source_url приложение строит переход к исходному моменту; сам поиск работает с локальными файлами.

Для самостоятельного построения индекса из уже сохранённого видео предусмотрен аргумент --local-video. Полный код получения видео, индексирования и интерфейса приведён ниже. Команда с --local-video читает указанный файл и не выполняет загрузку ролика.

```
python scripts/rutube_download.py https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/ \
    --local-video data/rutube/download/video.mp4 --out data/rutube --step 2
```

7.4. Полные листинги

Листинг — rutube_download.py. Полный исходный текст файла rutube_download.py.

```
"""User-run RuTube downloader. Importing this file performs no request."""
from pathlib import Path
from urllib.parse import urlsplit
import argparse, json, re

def validate_url(url):
    part=urlsplit(url)
    if part.scheme!='https' or (part.hostname or '').lower() not in
{'rutube.ru','www.rutube.ru'}:
        raise ValueError('Нужна HTTPS-ссылка RuTube')
    match=re.fullmatch(r'/video/([0-9a-fA-F]{32})/?',part.path)
    if not match:raise ValueError('Нужна ссылка на отдельный публичный ролик')
    return f'https://rutube.ru/video/{match.group(1).lower()}/'
def download(url,destination):
    from yt_dlp import YoutubeDL
    destination=Path(destination);destination.mkdir(parents=True,exist_ok=True)
    options={'format':'bestvideo[height<=720]+bestaudio/best[height<=720]/
best','outtmpl':str(destination/'%(id)s.%
(ext)s'),'merge_output_format':'mp4','noplaylist':True,'socket_timeout':30,'retries':2,'overwrite
s':False}
    with YoutubeDL(options) as loader:
```

```
info=loader.extract_info(validate_url(url),download=True);path=Path(loader.prepare_filename(info)
).with_suffix('.mp4')
        if not path.exists():path=Path(loader.prepare_filename(info))

(destination/'source.json').write_text(json.dumps({'url':url,'title':info.get('title'),'duration'
:info.get('duration'),'video_path':str(path)},ensure_ascii=False,indent=2))
        return path
def main():
    p=argparse.ArgumentParser();p.add_argument('url');p.add_argument('--out',default='data/
rutube');p.add_argument('--step',type=float,default=2);p.add_argument('--local-
video');a=p.parse_args();url=validate_url(a.url);out=Path(a.out)
    video=Path(a.local_video) if a.local_video else download(url,out/'download')
    from video_index import index_video

rows=index_video(video,out/'index',a.step,url);print('Индекс:',out/'index','Кадров:',len(rows))
if __name__=='__main__':main()
```

Листинг — video_index.py. Полный исходный текст файла video_index.py.

```
"""Create and search a CLIP index by sequentially reading a local video."""
import argparse
import json
from pathlib import Path
import cv2
import numpy as np
import torch
from PIL import Image
from torch.nn import functional as F
from transformers import CLIPModel, CLIPProcessor

def model():
    torch.set_num_threads(2)
    return (CLIPModel.from_pretrained('openai/clip-vit-base-patch32',
use_safetensors=True).eval(),
            CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32'))

def index_video(video, out, step=2., source_url=''):
    if step <= 0:
        raise ValueError('step must be positive')
    out=Path(out)
    frames=out/'frames'
    frames.mkdir(parents=True,exist_ok=True)
    cap=cv2.VideoCapture(str(video))
    fps=cap.get(cv2.CAP_PROP_FPS)
    if not cap.isOpened() or fps <= 0:
        cap.release()
        raise ValueError('Video cannot be opened or has invalid FPS')
    net,processor=model()
    stride=max(1,round(fps*step))
    rows=[];vectors=[];number=0
    try:
        while True:
            ok,frame=cap.read()
            if not ok:
                break
            if number % stride == 0:
                path=frames/f'frame_{number:06d}.jpg'
                cv2.imwrite(str(path),frame)
                image=Image.fromarray(cv2.cvtColor(frame,cv2.COLOR_BGR2RGB))
                with torch.no_grad():

vector=F.normalize(net.get_image_features(**processor(images=image,return_tensors='pt')),dim=1)
[0]
                vectors.append(vector.numpy())
                rows.append({'frame':number,'seconds':number/fps,
                             'path':str(path.relative_to(out)),'source_url':source_url})
            number+=1
    finally:
        cap.release()
    if not rows:
        raise ValueError('Video contains no decoded frames')
    np.save(out/'vectors.npy',np.stack(vectors))
    metadata={'fps':fps,'count':number,'step_s':step,'source_url':source_url,'records':rows}
```

```
(out/'index.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8')
    return rows

def search(directory, query, net, processor, image=None, gap=6., k=3):
    directory=Path(directory)
    metadata=json.loads((directory/'index.json').read_text(encoding='utf-8'))
    vectors=np.load(directory/'vectors.npy')
    if vectors.ndim!=2 or len(vectors)!=len(metadata['records']):
        raise ValueError('Index metadata and vectors disagree')
    with torch.no_grad():
        if image is None:

encoded=net.get_text_features(**processor(text=[query],padding=True,return_tensors='pt'))
        else:

encoded=net.get_image_features(**processor(images=image.convert('RGB'),return_tensors='pt'))
        vector=F.normalize(encoded,dim=1)[0].numpy()
    scores=vectors@vector
    chosen=[]
    for i in np.argsort(-scores):
        if any(abs(metadata['records'][i]['seconds']-metadata['records'][j]['seconds'])<gap
               or vectors[i]@vectors[j]>.98 for j in chosen):
            continue
        chosen.append(int(i))
        if len(chosen)==k:
            break
    return [dict(metadata['records'][i],cosine=float(scores[i])) for i in chosen]

if __name__=='__main__':
    parser=argparse.ArgumentParser(__doc__)
    parser.add_argument('video')
    parser.add_argument('--out',default='data/video_index')
    parser.add_argument('--step',type=float,default=2)
    parser.add_argument('--source-url',default='')
    args=parser.parse_args()
    print('Indexed',len(index_video(args.video,args.out,args.step,args.source_url)))
```

Листинг — app.py. Полный исходный текст файла app.py.

```
"""Streamlit search by text or example image in a prepared local index."""
from pathlib import Path
from urllib.parse import urlsplit, urlunsplit, parse_qsl, urlencode
import streamlit as st
from PIL import Image
from video_index import model, search

@st.cache_resource
def get_model():
    return model()

st.set_page_config(page_title='Поиск кадров в микроскопическом видео',layout='wide')
st.title('Поиск кадров в микроскопическом видео')
directory=Path(st.text_input('Каталог индекса','data/video_index'))
query=st.text_input('Запрос','Treponema pallidum spirochetes moving in a microscope field')
uploaded=st.file_uploader('Или изображение для поиска',type=['jpg','jpeg','png'])
gap=st.slider('Минимальный интервал между результатами, с',0,15,6)
if st.button('Найти'):
    if not (directory/'index.json').exists() or not (directory/'vectors.npy').exists():
        st.error('Индекс не найден. Укажите каталог с index.json и vectors.npy.')
        st.stop()
    net,processor=get_model()
    image=Image.open(uploaded).convert('RGB') if uploaded is not None else None
    hits=search(directory,query,net,processor,image=image,gap=gap)
    if not hits:
        st.info('Для заданных условий подходящие кадры отсутствуют.')
    for row in hits:
        st.image(str(directory/row['path']),width=480)
        st.write(f"Время: {row['seconds']:.2f} с; сходство: {row['cosine']:.3f}")
        if row.get('source_url'):
            parts=urlsplit(row['source_url'])
            if parts.hostname in {'rutube.ru','www.rutube.ru'}:
                params=dict(parse_qsl(parts.query));params['t']=str(int(row['seconds']))

url=urlunsplit((parts.scheme,parts.netloc,parts.path,urlencode(params),parts.fragment))
```

```
else:
    url=row['source_url']
st.link_button('Открыть источник',url)
```

8. Автоматическая подготовка разметки для CVAT

8.1. Маски, полигоны и скелеты

Вход — основное изображение и сохранённая цветовая маска. Цель локальной подготовки — представить шесть крупных видимых фрагментов в форматах, пригодных для редактирования в CVAT. Для каждого объекта записываются рамка, площадь, полигон и схема из трёх точек. Контур сохраняется в полном виде и после упрощения алгоритмом Дугласа — Пекера с допуском 2 пикселя.

Алгоритм выполнения. 1) Прочитать RGB-изображение и маску. 2) Найти связные компоненты площадью более 450 пикселей. 3) Для каждой компоненты извлечь внешний контур. 4) Создать полный полигон и упрощённый полигон. 5) Построить осевую линию и три точки. 6) Сформировать COCO JSON и поместить его вместе с изображением в ZIP. 7) Сохранить CVAT XML со скелетами. 8) Проверить число объектов и вершин в архиве.

Таблица 19

Вариант COCO	**Объектов**	**Вершин полигонов**
full	6	5662
simplified	6	370

Полный вариант содержит 5662 вершины, упрощённый — 370 при одинаковых шести объектах. Число вершин уменьшается на 93,47 %. Сокращение облегчает ручное редактирование; степень упрощения выбирается так, чтобы сохранить изгибы и не соединить близкие участки спирали. В COCO записаны images, categories и annotations; в каждой аннотации присутствуют image_id, category_id, bbox, area, iscrowd и segmentation.



Рисунок 22 — Изображение, маска и скелеты, использованные для формирования файлов разметки

Таблица 20

Файл	**Содержимое**
results/final/cvat/full_coco.zip	Исходное изображение и полные полигоны COCO
results/final/cvat/simplified_coco.zip	То же изображение и упрощённые полигоны
results/final/cvat/annotations.xml	Полигоны и три точки каждого скелета

8.2. Порядок использования в CVAT и сценарии AutoCVAT

Для загрузки COCO создаётся проект с меткой treponema и задача с treponema_primary.jpg. Через Actions → Upload annotations выбирается COCO 1.0 и нужный ZIP. Имена изображения и категории должны совпадать с задачей. Для XML дополнительно задаётся скелет treponema_pose с дочерними точками start, middle, end. После загрузки просматриваются края изображения, пересечения и правильность порядка точек.

Методичка рассматривает три сценария AutoCVAT: сегментационную модель с полными и упрощёнными контурами, объединение классов и zero-shot YOLO-World с сохранённым словарём. В представленных локальных архивах маска получена цветовым методом; это определяет происхождение полигонов. Для нейросетевого сценария используются веса модели сегментации и конфигурация names, confs, iou, imgsz, segment, minimize_points. При объединении классов исходные category_id отображаются в одну целевую метку до сохранения архива.

YOLO-World в опыте раздела 6 возвратил ноль рамок при conf = 0,1. Такой выход соответствует пустому списку annotations для выбранной

картинки и словаря. Число объектов, автоматически получаемых разными моделями, сравнивается до ручного исправления. Выход текущего раздела — локальные архивы и XML, подготовленные для указанного порядка загрузки и проверки в CVAT.

8.3. Полная программа экспорта

Листинг — run_annotations.py. Полный исходный текст файла run_annotations.py.

```
"""Produce measured skeleton landmarks and COCO/CVAT polygon exports."""
from experiment_common import *
import zipfile
import shutil
import xml.etree.ElementTree as ET

def main():
    source=rgb(ROOT/'data/raw/treponema_primary.jpg')
    mask=cv2.imread(str(ROOT/'results/primary_mask.png'),0)
    count,labels,stats,_=cv2.connectedComponentsWithStats((mask>0).astype('uint8'))
    shown=source.copy();records=[];annotations=[]
    xml=ET.Element('annotations');ET.SubElement(xml,'version').text='1.1'

node=ET.SubElement(xml,'image',id='0',name='treponema_primary.jpg',width=str(source.shape[1]),hei
ght=str(source.shape[0]))
    for label in range(1,count):
        x,y,w,h,area=map(int,stats[label])
        if area<=450:continue
        points,path=skeleton_points(labels==label)
        for a,b in
zip(path[:-1],path[1:]):cv2.line(shown,tuple(a.astype(int)),tuple(b.astype(int)),(30,255,30),1)
        for i,point in enumerate(points):
            cv2.circle(shown,tuple(point.astype(int)),5,(255,220,0),-1)
            cv2.putText(shown,str(i),tuple(point.astype(int)+[4,-4]),0,.6,(0,0,0),2)

contour=max(cv2.findContours((labels==label).astype('uint8'),cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_N
ONE)[0],key=cv2.contourArea)
        simplified=cv2.approxPolyDP(contour,2,True)
        records.append({'id':label,'area_px':area,'keypoints':points.tolist(),
                        'arc_length_px':float(np.linalg.norm(np.diff(path,axis=0),axis=1).sum()),
                        'contour_points':len(contour),'simplified_points':len(simplified)})
        annotations.append({'id':label,'image_id':1,'category_id':1,'bbox':[x,y,w,h],
                            'area':area,'iscrowd':0,'segmentation':
[contour.reshape(-1).tolist()]})
        ET.SubElement(node,'polygon',label='treponema',occluded='0',source='auto',
                      points=';'.join(f'{a},{b}' for a,b in simplified[:,0,:]))
        ET.SubElement(node,'skeleton',label='treponema_pose',source='auto')
        skeleton=node[-1]
        for name,p in zip(['start','middle','end'],points):
            ET.SubElement(skeleton,'points',label=name,points=f'{p[0]},
{p[1]}',occluded='0',outside='0')
    panel('skeleton_landmarks.png',[source,mask,shown],['Исходник','Цветовая маска','Точки вдоль
осевой линии'])
    save('skeleton_landmarks.json',records)
    out=OUT/'cvat';out.mkdir(exist_ok=True)

ET.indent(xml);ET.ElementTree(xml).write(out/'annotations.xml',encoding='utf-8',xml_declaration=T
rue)
    totals={}
    for variant in ['full','simplified']:
        payload={'images':
[{'id':1,'file_name':'treponema_primary.jpg','width':source.shape[1],'height':source.shape[0]}],
                 'categories':
[{'id':1,'name':'treponema'}],'annotations':json.loads(json.dumps(annotations))}
        if variant=='simplified':
            for ann in payload['annotations']:
                contour=np.array(ann['segmentation'][0],np.int32).reshape(-1,1,2)
                ann['segmentation']=[cv2.approxPolyDP(contour,2,True).reshape(-1).tolist()]
```

```
        points=sum(len(a['segmentation'][0])//2 for a in payload['annotations'])
        totals[variant]={'objects':len(payload['annotations']),'polygon_points':points}
        with zipfile.ZipFile(out/f'{variant}_coco.zip','w',zipfile.ZIP_DEFLATED) as archive:
            archive.writestr('annotations/instances_default.json',json.dumps(payload))
            archive.write(ROOT/'data/raw/treponema_primary.jpg','images/treponema_primary.jpg')
    save('cvat_actual.json',totals)
    print(totals,flush=True)

if __name__=='__main__':
    main()
```

9. Учёт экспериментов в MLflow

9.1. Организация запусков

MLflow связывает параметры, метрики и модели с идентификатором запуска. В эксперименте Treponema_LR4_measured используются 512-мерные фиксированные признаки ResNet-18 из раздела 4. На одинаковом разбиении сравниваются деревья решений и полносвязные сети. Это вариант классификаторов на нейросетевых признаках; вход MLP имеет размер 512, а не развёрнутые пиксели изображения.

Алгоритм выполнения. 1) Задать локальный SQLite backend и имя эксперимента. 2) Для каждой конфигурации начать отдельный run. 3) Записать гиперпараметры. 4) Обучить классификатор на train. 5) Записать проверочные и тестовые метрики. 6) Сохранить модель средствами MLflow. 7) Зарегистрировать две версии выбранных деревьев. 8) Загрузить модели по URI и сравнить предсказания с исходными.

Сетка деревьев: max_depth ∈ {2, 4, 8, 16}, min_samples_split ∈ {2, 10}; всего восемь запусков. Для MLP используются скрытые размеры 32, 64 и 128 с Dropout 0; 0,2; 0,5 в пяти конфигурациях. MLP обучается 60 эпох с Adam и learning rate 0,005. На каждой эпохе записывается validation accuracy, а после обучения — test macro-F1. Macro-F1 усредняет F1 трёх классов с одинаковым весом.

9.2. Таблицы запусков и кривые

Таблица 21

Запуск дерева	**Validation macro-F1**	**Test macro-F1**
tree_2_2	0.153846	0.323529
tree_2_10	0.153846	0.323529
tree_4_2	0.249084	0.365142
tree_4_10	0.249084	0.323529
tree_8_2	0.249084	0.365142
tree_8_10	0.249084	0.323529
tree_16_2	0.249084	0.365142

Запуск дерева	Validation macro-F1	Test macro-F1
tree_16_10	0.249084	0.323529

Таблица 22

Запуск MLP	Последняя val accuracy	Test macro-F1	Повторная загрузка
mlp_32_0	0.400000	0.434174	Совпадение
mlp_64_0	0.600000	0.534093	Совпадение
mlp_64_0.2	0.533333	0.532601	Совпадение
mlp_128_0.2	0.466667	0.592075	Совпадение
mlp_128_0.5	0.533333	0.592075	Совпадение

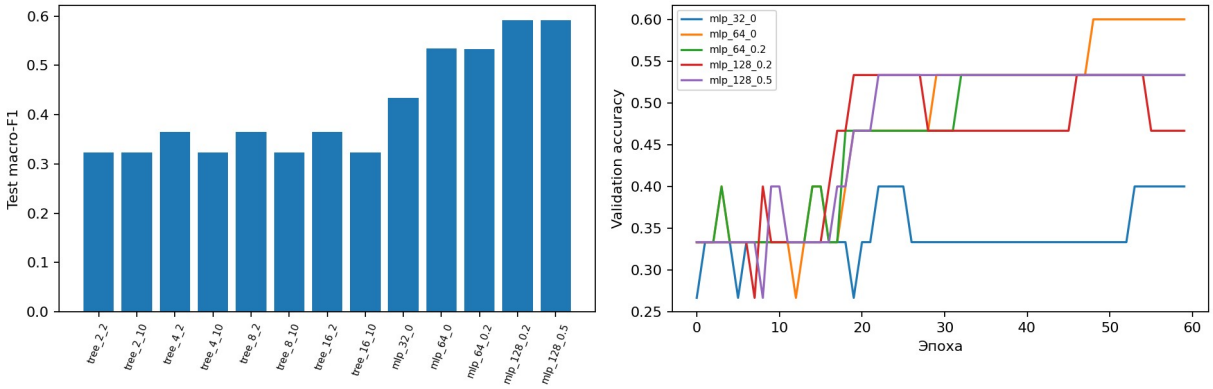


Рисунок 23 — Сводка 13 запусков MLflow и сохранённые кривые validation accuracy пяти MLP

Деревья глубины 4, 8 и 16 достигают одинакового validation macro-F1 0,249084. При min_samples_split = 2 их test macro-F1 равен 0,365142. Повторение результата на большей глубине показывает, что увеличение ограничения глубины само по себе не изменяет выбранное разбиение. Кривые MLP отражают колебания проверки на 15 изображениях; изменение одного предсказания меняет accuracy примерно на 0,0667.

9.3. Реестр моделей и проверка загрузки

Таблица 23

Имя	Версия	Совпадение предсказаний
Treponema_ResNet_Tree	1	Да
Treponema_ResNet_Tree	2	Да

В реестре Treponema_ResNet_Tree сохранены две версии. После загрузки по URI models:/Treponema_ResNet_Tree/1 и models:/Treponema_ResNet_Tree/2 предсказания на тестовых признаках совпадают с предсказаниями исходных объектов моделей. Аналогичная

проверка выполнена для пяти MLP через mlflow.pyfunc. Проверяется последовательность «обучение — сохранение — загрузка — предсказание».

Таблица 24

Запуск	**Run ID**
tree_2_2	00883bc5a43548029076a983be5ace86
tree_2_10	f4cea75558d948699d0db3022a554cbe
tree_4_2	da8a306a15324e369be089b626e04eb3
tree_4_10	cd376cb5fdfe4265b32c0a5a3a20d70e
tree_8_2	a0dc7cc86f7c4652a083feb90b5f34ff
tree_8_10	242231592a5b47139fdb22e458e1a86d
tree_16_2	53af34e0247644b384135d58a20c6409
tree_16_10	578019306f1b401eb2bf86e910055c80
mlp_32_0	d3cfc32c25814ea9821477519324d7d1
mlp_64_0	e5eedcdc6d364bde9a2c3f10819ee973
mlp_64_0.2	b65b7ef9ef964153b948aeb7770e9945
mlp_128_0.2	a9150e2cc98e4f0782ac8f65e6eb49dd
mlp_128_0.5	2558f0b3016d4f179f2645d0550863f2

Идентификаторы позволяют связать строку таблицы с параметрами и артефактами запуска. Для просмотра локальной базы используется команда mlflow ui с backend-store-uri, указывающим на .cache/mlflow.db. График выше построен по сохранённым численным значениям запусков; JSON содержит полные последовательности значений по эпохам.

9.4. Полная программа регистрации экспериментов

Листинг — run_learning.py. Полный исходный текст файла run_learning.py.

```
"""CLIP/ResNet retrieval, triplet projection, 8 trees and 5 MLP runs in MLflow."""
from experiment_common import *
from PIL import Image
from torch import nn
from torch.nn import functional as F
from torchvision.models import resnet18, ResNet18_Weights
from transformers import CLIPModel, CLIPProcessor
from sklearn.metrics import accuracy_score, balanced_accuracy_score, f1_score, roc_auc_score,
confusion_matrix
from sklearn.tree import DecisionTreeClassifier
import mlflow
import mlflow.sklearn
import mlflow.pytorch
import itertools

def main():
    # Near-duplicate colour variants t04/t05 are kept together in train.
    assignments=[('t01.jpg',0,'train'),('t02.jpg',0,'train'),('t04.jpg',0,'train'),
                 ('t05.jpg',0,'train'),('t03.png',0,'val'),('t06.jpg',0,'test'),
                 ('treponema_primary.jpg',0,'test'),
```

```
                ('s01.jpg',1,'train'),('s05.jpg',1,'train'),('s06.jpg',1,'train'),
                ('s03.jpg',1,'val'),('s04.jpg',1,'test'),('s09.jpg',1,'test'),
                ('s07.jpg',2,'train'),('s08.jpg',2,'val'),('s02.jpg',2,'test')]
    gallery=ROOT/'data/final_gallery';gallery.mkdir(parents=True,exist_ok=True)
    images=[];rows=[]
    for filename,label,split in assignments:
        path=(LEGACY if filename.startswith('s') else ROOT)/'data/raw'/filename
        image=Image.open(path).convert('RGB');w,h=image.size
        for j,box in enumerate([(0,0,w,h),(0,0,w//2,h//2),(w//2,0,w,h//2),
                                (0,h//2,w//2,h),(w//2,h//2,w,h)]):
            patch=image.crop(box).resize((224,224));name=Path(filename).stem+f'_{j}.jpg'
            patch.save(gallery/name);images.append(patch)
            rows.append({'file':name,'source':filename,'label':label,'split':split})
    save('gallery.json',rows)
    clip=CLIPModel.from_pretrained('openai/clip-vit-base-patch32',use_safetensors=True).eval()
    processor=CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
    clip_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):

clip_vectors.append(F.normalize(clip.get_image_features(**processor(images=images[i:i+8],
                                    return_tensors='pt')),dim=1).numpy())
    clip_vectors=np.concatenate(clip_vectors)

weights=ResNet18_Weights.DEFAULT;resnet=resnet18(weights=weights).eval();resnet.fc=nn.Identity()
    resnet_vectors=[]
    with torch.no_grad():
        for i in range(0,len(images),8):
            resnet_vectors.append(F.normalize(resnet(torch.stack([weights.transforms()(im)
                                for im in images[i:i+8]])),dim=1).numpy())
    resnet_vectors=np.concatenate(resnet_vectors)
    np.save(OUT/'clip_vectors.npy',clip_vectors);np.save(OUT/'resnet_vectors.npy',resnet_vectors)
    labels=np.array([r['label'] for r in rows]);splits=np.array([r['split'] for r in rows])

train=np.flatnonzero(splits=='train');val=np.flatnonzero(splits=='val');test=np.flatnonzero(split
s=='test')
    prompts=['Treponema pallidum spiral spirochetes','rod shaped bacteria','round cocci
bacteria']
    with
torch.no_grad():q=F.normalize(clip.get_text_features(**processor(text=prompts,padding=True,
                                                return_tensors='pt')),dim=1).numpy()
    hits=[];panels=[];titles=[]
    for prompt,vector in zip(prompts,q):
        scores=clip_vectors@vector;ids=np.argsort(-scores)[:3]
        hits.append({'query':prompt,'hits':[{'file':rows[i]['file'],'cosine':float(scores[i])}
for i in ids]})
        for i in ids:panels.append(np.array(images[i]));titles.append(f'{prompt[:25]}\n{rows[i]
["file"]}: {scores[i]:.3f}')
    panel('retrieval_text.png',panels,titles)
    def precision(vectors):
        values=[]
        for i in test:
            candidates=[j for j in test if rows[i]['source']!=rows[j]['source']]
            order=sorted(candidates,key=lambda j:float(vectors[i]@vectors[j]),reverse=True)[:3]
            values.append(float(np.mean(labels[order]==labels[i])))
        return float(np.mean(values))
    query_index=next(i for i,r in enumerate(rows) if r['file']=='treponema_primary_0.jpg')
    candidates=[i for i,r in enumerate(rows) if r['source']!='treponema_primary.jpg']
    idx=sorted(candidates,key=lambda
i:float(resnet_vectors[query_index]@resnet_vectors[i]),reverse=True)[:3]
    panel('retrieval_image.png',[np.array(images[query_index])]+[np.array(images[i]) for i in
idx],
          ['Запрос']+[rows[i]['file'] for i in idx],cols=4)
    save('retrieval_actual.json',
{'samples':len(rows),'train':len(train),'val':len(val),'test':len(test),
       'classes':['Treponema','rods','cocci'],'text_queries':hits,
       'CLIP_precision_at_3_cross_source':precision(clip_vectors),
       'ResNet_precision_at_3_cross_source':precision(resnet_vectors)})

    x=torch.tensor(resnet_vectors,dtype=torch.float32)
    projection=nn.Sequential(nn.Linear(512,128),nn.ReLU(),nn.Linear(128,64))
    optimizer=torch.optim.Adam(projection.parameters(),lr=.001)
    rng=np.random.default_rng(7);history=[];best=None;best_score=-1
    def pairs(ids):
        # Pairs share patches and may share a source; they are not independent samples.
        return list(itertools.combinations(ids,2))
    def evaluate(z,ids,threshold=None):
        pp=pairs(ids);truth=np.array([labels[i]==labels[j] for i,j in
```

```
pp]);scores=np.array([z[i]@z[j] for i,j in pp])
        if threshold is None:
            choices=np.linspace(-1,1,401)
            predictions=scores[:,None]>=choices
            balanced=(predictions[truth].mean(0)+(~predictions[~truth]).mean(0))/2
            threshold=choices[int(np.argmax(balanced))]
        return
{'threshold':float(threshold),'balanced_accuracy':float(balanced_accuracy_score(truth,scores>=thr
eshold)),
                'AUC':float(roc_auc_score(truth,scores)),'pairs':len(pp)}
    for epoch in range(100):
        triples=[]
        for i in train:
            positives=[j for j in train if labels[j]==labels[i] and j!=i]
            negatives=[j for j in train if labels[j]!=labels[i]]
            triples.append([i,rng.choice(positives),rng.choice(negatives)])
        t=np.array(triples);z=F.normalize(projection(x),dim=1)
        loss=F.triplet_margin_loss(z[t[:,0]],z[t[:,1]],z[t[:,2]],margin=.5)
        optimizer.zero_grad();loss.backward();optimizer.step()
        with torch.no_grad():vectors=F.normalize(projection(x),dim=1).numpy()
        validation=evaluate(vectors,val)
        if validation['balanced_accuracy']>best_score:
            best_score=validation['balanced_accuracy'];best={k:v.detach().clone() for k,v in
projection.state_dict().items()}
        history.append(float(loss.detach()))
    projection.load_state_dict(best)
    with torch.no_grad():learned=F.normalize(projection(x),dim=1).numpy()
    baseline=evaluate(resnet_vectors,val);after=evaluate(learned,val)
    metric={'base':evaluate(resnet_vectors,test,baseline['threshold']),
            'triplet':evaluate(learned,test,after['threshold']), 'train_loss':history}
    torch.save(projection.state_dict(),OUT/'triplet_head.pt');save('metric_actual.json',metric)

fig,ax=plt.subplots(figsize=(8,3));ax.plot(range(1,101),history);ax.set(xlabel='Эпоха',ylabel='Tr
iplet loss')
    fig.tight_layout();fig.savefig(OUT/'triplet_learning.png',dpi=160);plt.close(fig)

    mlflow.set_tracking_uri('sqlite:///'+str(CACHE/'mlflow.db'))
    mlflow.set_experiment('Treponema_LR4_measured')
    mlflow_runs=[];tree_models=[]
    for depth in [2,4,8,16]:
        for minimum in [2,10]:

estimator=DecisionTreeClassifier(max_depth=depth,min_samples_split=minimum,random_state=7).fit(re
snet_vectors[train],labels[train])

pred=estimator.predict(resnet_vectors[test]);vpred=estimator.predict(resnet_vectors[val])
            with mlflow.start_run(run_name=f'tree_{depth}_{minimum}') as run:
                mlflow.log_params({'depth':depth,'min_samples_split':minimum,'backbone':'ResNet18
frozen'})
                scores={'val_f1':float(f1_score(labels[val],vpred,average='macro')),
                        'test_f1':float(f1_score(labels[test],pred,average='macro'))}
                mlflow.log_metrics(scores)
                info=mlflow.sklearn.log_model(estimator,name='model',
                         skops_trusted_types=['sklearn.tree._tree.Tree'])
                mlflow_runs.append({'name':f'tree_{depth}
_{minimum}','run_id':run.info.run_id,**scores})
                tree_models.append((scores['val_f1'],info.model_uri,estimator))
    network_runs=[]
    for hidden,dropout in [(32,0),(64,0),(64,.2),(128,.2),(128,.5)]:
        torch.manual_seed(7)

net=nn.Sequential(nn.Linear(512,hidden),nn.ReLU(),nn.Dropout(dropout),nn.Linear(hidden,3))
        optim=torch.optim.Adam(net.parameters(),lr=.005);ys=torch.tensor(labels,dtype=torch.long)
        values=[]
        with mlflow.start_run(run_name=f'mlp_{hidden}_{dropout}') as run:
            mlflow.log_params({'hidden':hidden,'dropout':dropout,'epochs':60,'backbone':'ResNet18
frozen'})
            for epoch in range(60):

net.train();loss=F.cross_entropy(net(x[train]),ys[train]);optim.zero_grad();loss.backward();optim
.step()
                net.eval()
                with torch.no_grad():v=net(x[val]).argmax(1).numpy()
                score=float(accuracy_score(labels[val],v));values.append(score)
                mlflow.log_metric('val_accuracy',score,step=epoch)
            with torch.no_grad():pred=net(x[test]).argmax(1).numpy()

score=float(f1_score(labels[test],pred,average='macro'));mlflow.log_metric('test_f1',score)
```

```
            info=mlflow.pytorch.log_model(net,name='model',input_example=resnet_vectors[test])
            loaded=mlflow.pyfunc.load_model(info.model_uri)
            restored_prediction=loaded.predict(resnet_vectors[test])
            same=bool(np.array_equal(np.asarray(restored_prediction).argmax(1),pred))
            network_runs.append({'name':f'mlp_{hidden}_{dropout}','run_id':run.info.run_id,
                                 'test_f1':score,'val_history':values,'reload_identical':same})
    registered=[]
    for _,uri,estimator in sorted(tree_models,key=lambda t:t[0],reverse=True)[:2]:
        version=mlflow.register_model(uri,'Treponema_ResNet_Tree')
        restored=mlflow.sklearn.load_model(f'models:/Treponema_ResNet_Tree/{version.version}')

identical=bool(np.array_equal(restored.predict(resnet_vectors[test]),estimator.predict(resnet_vec
tors[test])))

registered.append({'name':version.name,'version':version.version,'reload_identical':identical})
    save('mlflow_actual.json',
{'trees':mlflow_runs,'networks':network_runs,'registered':registered})
    fig,axes=plt.subplots(1,2,figsize=(12,4))
    runs=mlflow_runs+network_runs;axes[0].bar(range(len(runs)),[r['test_f1'] for r in runs])
    axes[0].set_xticks(range(len(runs)),[r['name'] for r in
runs],rotation=70,fontsize=7);axes[0].set_ylabel('Test macro-F1')
    for r in network_runs:axes[1].plot(r['val_history'],label=r['name'])
    axes[1].set(xlabel='Эпоха',ylabel='Validation accuracy');axes[1].legend(fontsize=7)
    fig.tight_layout();fig.savefig(OUT/'mlflow_actual.png',dpi=160);plt.close(fig)
    print('Completed retrieval, metric learning and 13 MLflow runs',flush=True)

if __name__=='__main__':
    main()
```

10. Полный цикл YOLO-pose

10.1. Набор, разметка и параметры обучения

Вход — шесть видимых компонент основной иллюстрации и три точки каждой осевой линии. Вокруг компоненты формируется серый фон со значением 114, фрагмент масштабируется и сдвигается на холсте 192 × 192. Первые четыре компоненты дают по восемь вариантов для train, пятая и шестая — по четыре варианта для validation и test. Разметка состоит из рамки и трёх видимых точек с нормированными координатами.

Алгоритм выполнения. 1) Выделить компоненты и точки. 2) Распределить исходные компоненты по частям набора. 3) Сформировать изображения и строки YOLO-pose. 4) Записать dataset.yaml. 5) Обучить YOLOv8s-pose и YOLO11s-pose с одинаковыми параметрами. 6) Загрузить best.pt каждого запуска. 7) Выполнить val на test. 8) Измерить расстояния точек и PCK, сохранить предсказания и CSV по эпохам.

Таблица 25

Параметр	**Значение**
Размер входа	192 × 192
Эпох	20
Batch	8
Устройство	CPU
Оптимизатор	AdamW
Начальный learning rate	0,001
Seed	7
degrees / flipud / mosaic	0 / 0 / 0
fliplr / translate / scale	0,5 / 0,1 / 0,1
kpt_shape / flip_idx	[3, 3] / [2, 1, 0]

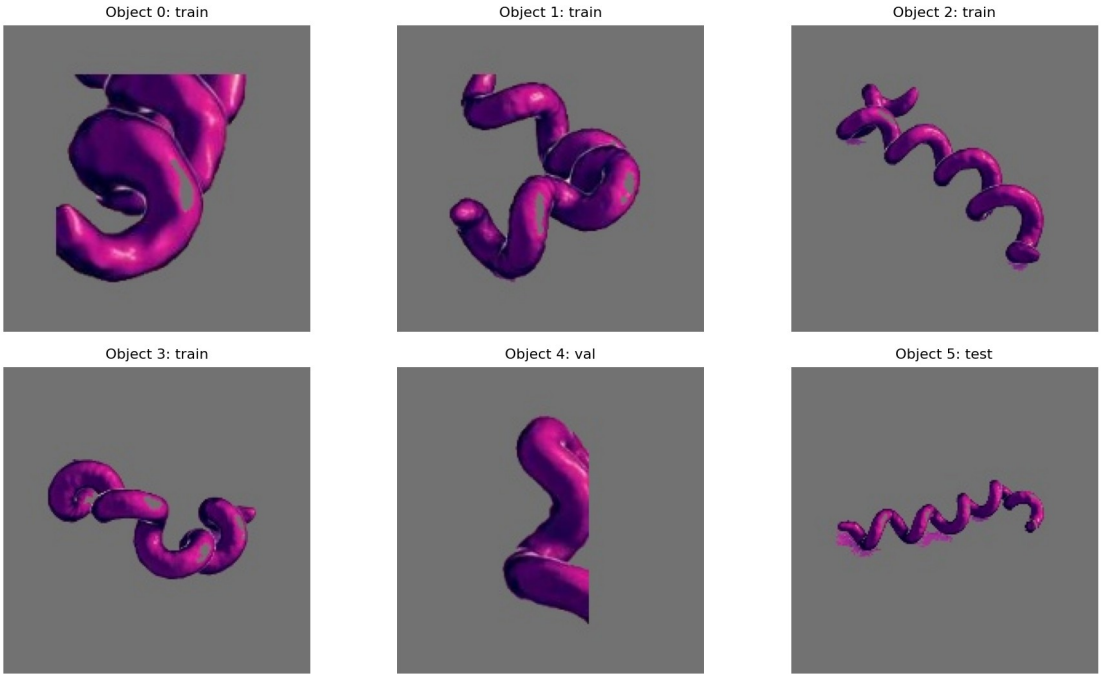


Рисунок 24 — Шесть исходных компонент и их распределение между train, validation и test

Поворот и вертикальное отражение выключены, поскольку фиксированный порядок крайних точек должен сохранять смысл. Горизонтальное отражение согласовано с flip_idx. Все точки получены из видимого скелета и имеют v = 2. Для расширенной разметки невидимой точки с v = 0 её координаты записываются нулями, чтобы исключить ложный вклад в геометрическую оценку.

10.2. Метрики и изображения предсказаний

Таблица 26

Модель	**mAP50(P)**	**mAP50–95(P)**	**mAP50–95(B)**	**Ошибка, px**	**PCK@0,1**
v8s	0.757658	0.077512	0.630000	49.827	0.000
11s	0.248750	0.049750	0.884875	59.717	0.000

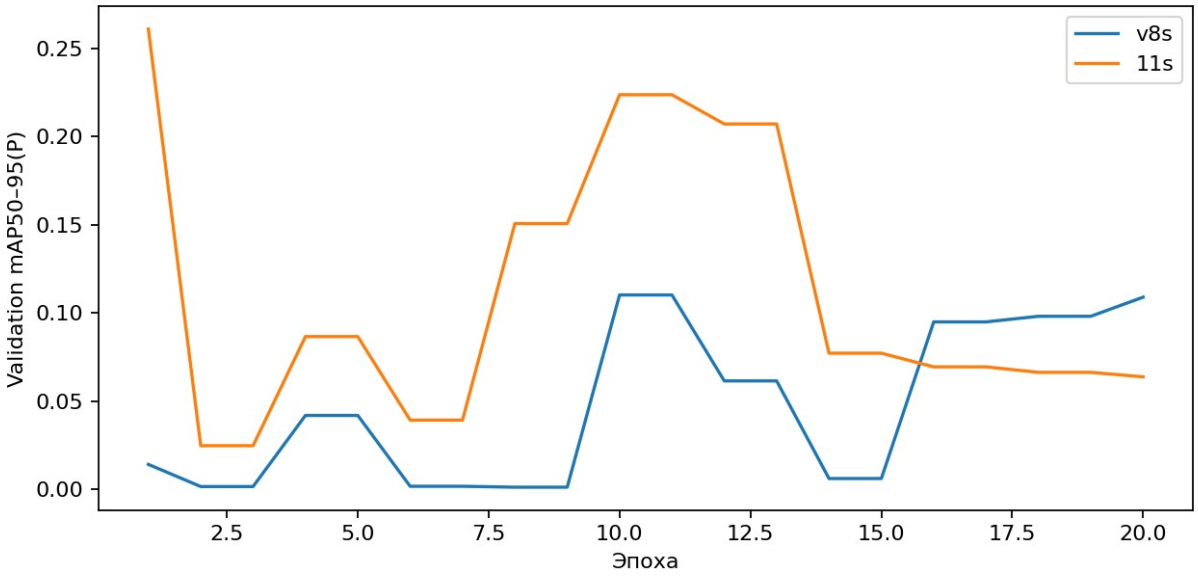


Рисунок 25 — Проверочная mAP50–95(P) по эпохам для YOLOv8s-pose и YOLO11s-pose

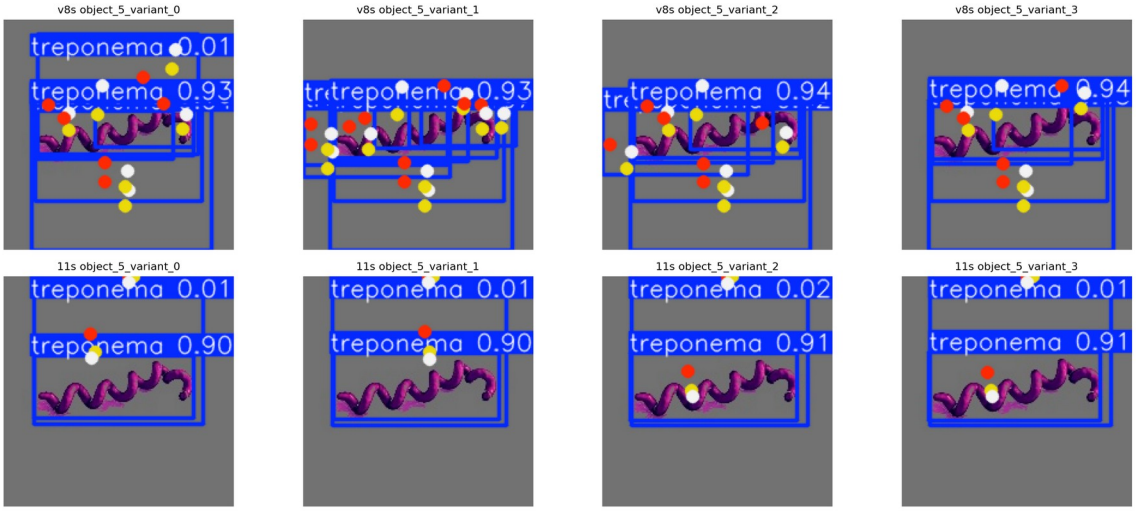


Рисунок 26 — Предсказания двух обученных pose-моделей на четырёх тестовых вариантах

Индекс P относится к ключевым точкам, B — к рамкам. mAP50(P) проверяет совпадение по OKS при пороге 0,5; mAP50–95(P) усредняет более строгий набор порогов. YOLOv8s получает 0,757658 по mAP50(P) и 0,077512 по mAP50–95(P). YOLO11s получает 0,248750 и 0,049750 соответственно. Для рамок значение mAP50–95(B) выше у YOLO11s: 0,884875 против 0,630000.

Средняя ошибка точки измеряется на холсте 192 × 192 для наиболее уверенной найденной рамки: 49,827 px у v8s и 59,717 px у 11s. PCK@0,1 использует допуск 10 % длины хорды между крайними истинными точками и

равен нулю для обеих моделей. Пропусков рамок при контрольном пороге conf = 0,01 — 0. Совместное чтение этих величин показывает, что локализация рамки и точность положения отдельных точек являются разными задачами.

Четыре тестовых изображения представляют варианты одной отложенной компоненты. Их результаты характеризуют перенос внутри исходной иллюстрации. Сохранённые best.pt позволяют продолжить обучение на новых размеченных изображениях; независимые микрофотографии и видеокадры дают более широкий контроль устойчивости, чем дополнительные преобразования одного фрагмента.

10.3. Выходные файлы и полный листинг

Таблица 27

Файл	**Назначение**
pose_v8s_best.pt; pose_11s_best.pt	Лучшие веса двух запусков
pose_v8s_history.csv; pose_11s_history.csv	Потери и метрики каждой эпохи
pose_training.json	Итоговая оценка test и геометрическая ошибка
data/final_pose/dataset.yaml	Пути к данным, имена классов и схема точек

Истории pose-обучения сохранены в CSV и сопоставлены на одном графике. В листинге интеграция Ultralytics с MLflow выключена; журнал классификационных экспериментов MLflow представлен в разделе 9. Это разделяет способ хранения результатов двух выполненных серий и позволяет воспроизвести их по соответствующим файлам.

Листинг — run_pose_training.py. Полный исходный текст файла run_pose_training.py.

```
"""Train and evaluate two pose models on grouped geometric Treponema fragments."""
from experiment_common import *
from ultralytics import YOLO, settings
import shutil

def main():
    settings.update({'mlflow':False})
    data=ROOT/'data/final_pose'
    source=cv2.imread(str(ROOT/'data/raw/treponema_primary.jpg'))
    mask=cv2.imread(str(ROOT/'results/primary_mask.png'),0)
    count,labels,stats,_=cv2.connectedComponentsWithStats((mask>0).astype('uint8'))
    objects=[i for i in range(1,count) if stats[i,4]>450]
    records=[]
    for index,label in enumerate(objects):
        x,y,w,h,area=map(int,stats[label]); yy,xx=np.nonzero(labels==label)
        kp,_=skeleton_points(labels==label)
```

```
        split='train' if index<4 else ('val' if index==4 else 'test')
        crop=source[y:y+h,x:x+w].copy()
        crop[labels[y:y+h,x:x+w]!=label]=114
        for variant in range(8 if split=='train' else 4):
            scale=(130+variant*4)/max(w,h)
            ww,hh=round(w*scale),round(h*scale)
            dx=(192-ww)//2+variant-3;dy=(192-hh)//2
            canvas=np.full((192,192,3),114,np.uint8)
            canvas[dy:dy+hh,dx:dx+ww]=cv2.resize(crop,(ww,hh))
            points=(kp-[x,y])*[ww/w,hh/h]+[dx,dy]
            box=[(dx+ww/2)/192,(dy+hh/2)/192,ww/192,hh/192]
            line=[0,*box]
            for point in points:line.extend([point[0]/192,point[1]/192,2])
            folder=data/'images'/split;folder.mkdir(parents=True,exist_ok=True)
            target=data/'labels'/split;target.mkdir(parents=True,exist_ok=True)
            name=f'object_{index}_variant_{variant}'
            cv2.imwrite(str(folder/(name+'.jpg')),canvas)
            (target/(name+'.txt')).write_text(' '.join(map(str,line))+'\n')
            records.append({'file':str((folder/(name+'.jpg')).relative_to(ROOT)),
                            'object':index,'split':split,'keypoints':points.tolist(),'box':box})
    yaml=data/'dataset.yaml'
    yaml.write_text(f'path: {data}\ntrain: images/train\nval: images/val\ntest: images/test\n'
                    'names: [treponema]\nkpt_shape: [3, 3]\nflip_idx: [2, 1, 0]\n')
    save('pose_dataset.json',records)
    gallery=[rgb(ROOT/r['file']) for r in records if r['file'].endswith('variant_0.jpg')]
    panel('pose_dataset.png',gallery,[f"Object {i}: {'train' if i<4 else ('val' if i==4 else
'test')}" for i in range(6)])
    metrics=[];predictions=[];titles=[]
    for weights,name in [('yolov8s-pose.pt','v8s'),('yolo11s-pose.pt','11s')]:
        net=YOLO(CACHE/weights)
        net.train(data=str(yaml),epochs=20,imgsz=192,batch=8,device='cpu',workers=0,
                  project=str(CACHE/'pose_training'),name=name,exist_ok=True,seed=7,
                  degrees=0,flipud=0,fliplr=.5,mosaic=0,translate=.1,scale=.1,
                  plots=True,verbose=False,optimizer='AdamW',lr0=.001)
        best=YOLO(net.trainer.best)

result=best.val(data=str(yaml),split='test',imgsz=192,batch=4,device='cpu',workers=0,plots=True)
        errors=[];pck=[];misses=0
        for record in [r for r in records if r['split']=='test']:
            r=best(ROOT/record['file'],conf=.01,imgsz=192,verbose=False)[0]
            predictions.append(cv2.cvtColor(r.plot(),cv2.COLOR_BGR2RGB));titles.append(name+'
'+Path(record['file']).stem)
            if not len(r.boxes):misses+=1;continue
            kp=r.keypoints.xy[0].cpu().numpy();truth=np.array(record['keypoints'])
            distance=np.linalg.norm(kp-truth,axis=1)
            errors.extend(distance.tolist())
            pck.extend((distance<.1*np.linalg.norm(truth[2]-truth[0])).tolist())
        metrics.append({'model':name,'epochs':20,'pose_map50':float(result.pose.map50),
                        'pose_map':float(result.pose.map),'box_map':float(result.box.map),
                        'mean_keypoint_error_px':float(np.mean(errors)) if errors else None,
                        'PCK_at_01':float(np.mean(pck)) if pck else None,'missed':misses})
        shutil.copy2(net.trainer.save_dir/'results.csv',OUT/f'pose_{name}_history.csv')
        shutil.copy2(net.trainer.best,OUT/f'pose_{name}_best.pt')
    save('pose_training.json',{'split_unit':'visible component in one illustration',

'train_objects':4,'val_objects':1,'test_objects':1,'models':metrics})
    panel('pose_predictions.png',predictions,titles,cols=4)
    import pandas as pd
    fig,ax=plt.subplots(figsize=(8,4))
    for name in ['v8s','11s']:
        frame=pd.read_csv(OUT/f'pose_{name}_history.csv');frame.columns=frame.columns.str.strip()
        ax.plot(frame['epoch'],frame['metrics/mAP50-95(P)'],label=name)
    ax.set(xlabel='Эпоха',ylabel='Validation mAP50–95(P)');ax.legend();fig.tight_layout()
    fig.savefig(OUT/'pose_learning.png',dpi=160);plt.close(fig)
    print(metrics,flush=True)

if __name__=='__main__':
    main()
```

11. Ответы на контрольные вопросы

11.1. Что означает ключевая точка и её видимость?

Ключевая точка имеет фиксированный смысл в схеме объекта. v = 2 означает видимость, v = 1 — закрытую точку с заданным положением, v = 0 — отсутствующую точку. Для трепонемы используются начало, середина и конец видимого фрагмента.

11.2. Что измеряет OKS?

OKS усредняет экспоненциальные оценки ошибки точек с учётом масштаба объекта и допуска для каждой точки. Как IoU для рамок, OKS определяет степень совпадения предсказания и разметки и используется для расчёта precision, recall и AP.

11.3. Почему flip_idx = [2, 1, 0]?

Горизонтальное отражение меняет местами концы и сохраняет центральную точку. Произвольный поворот и вертикальное отражение могут нарушать принятый порядок полюсов, поэтому для данной схемы эти преобразования выключены.

11.4. Чем различаются SOT-трекеры?

MIL обучает классификатор онлайн, MOSSE и KCF используют корреляционные фильтры, CSRT учитывает пространственную и канальную надёжность. GOTURN предсказывает положение объекта обученной сетью по предыдущему и текущему изображениям. Сходство соседних объектов и накопление ошибки обновления могут вызывать смену сопровождаемого объекта.

11.5. Почему усредняют векторы, а не углы?

Углы цикличны: среднее для 179° и −179° не должно давать 0°. Сначала усредняются горизонтальные и вертикальные компоненты, затем

направление вычисляется через atan2. Длина среднего вектора также отражает согласованность движения.

11.6. Что такое проблема апертуры?

На небольшом прямолинейном участке видна главным образом компонента движения поперёк границы. Лукас — Канаде объединяет уравнения для окрестности; участки с градиентами разных направлений, например углы, дают более определённое решение.

11.7. Почему GAN-сверхразрешение требует внимательной оценки?

Генеративная модель может добавлять правдоподобные детали, отсутствовавшие в исходных пикселях. В микроскопии такие детали нельзя автоматически трактовать как реальную структуру объекта. В лабораторной используются эталон, PSNR, SSIM и отдельная оценка OCR.

11.8. Для чего используется косинусное сходство?

Оно сравнивает направления признаковых векторов и после L2-нормировки не зависит от их исходного масштаба. Для единичных векторов квадрат евклидова расстояния равен 2 − 2cos, поэтому оба способа дают одинаковое ранжирование.

11.9. Зачем нормировка, трудные негативы и отдельная калибровка?

Нормировка контролирует масштаб эмбеддинга, трудные негативы дают ненулевой обучающий сигнал, а калибровочная выборка позволяет выбрать порог без использования тестовых ответов. Для проверки неизвестных классов требуется разбиение по классам; в выполненной серии используется разбиение по источникам внутри трёх групп.

11.10. Почему CLIP выбирает одно из заданных описаний?

При сравнении конечного списка всегда существует максимальный логит или cosine. Softmax нормирует только этот список и не добавляет автоматически класс «ничего из перечисленного». Поэтому нужен контроль состава описаний и при необходимости отдельный порог отказа.

11.11. Зачем удалять похожие кадры и как строится ссылка?

Соседние кадры часто повторяют одну сцену. Временной интервал и порог сходства уменьшают повторение результатов. К базовому URL вида https://rutube.ru/video/ID/ добавляется ?t=число_секунд.

11.12. Что содержит COCO-архив и зачем minimize_points?

Архив содержит изображения и JSON с перечнем изображений, категорий и аннотаций. Для сегментации записываются полигоны и рамки. Упрощение уменьшает число вершин и облегчает ручную правку; допуск выбирается с учётом масштаба и кривизны контура.

11.13. Что сохраняет MLflow и для чего реестр?

Запуск связывает параметры, метрики, историю по шагам, артефакты и модель. Реестр присваивает моделям имена и версии, позволяя явно загрузить нужную версию и проверить воспроизводимость предсказания.

11.14. Почему координаты отсутствующих точек обнуляют?

При v = 0 позиция не является заданной геометрической целью. Нулевые координаты согласуют отсутствие точки с форматом данных и исключают случайное использование координат как видимой разметки.

Заключение

В лабораторной работе построен конвейер анализа изображений Treponema pallidum и контрольных материалов. Подготовлены маски и три точки видимых фрагментов, выполнены сравнения готовых человеческих pose-моделей и обучение двух моделей со своей схемой. Контроль на людях показывает ожидаемую структуру выходов, а опыт на трепонеме — влияние области обучения и порога уверенности.

На микроскопическом видео сопоставлены MIL, MOSSE, KCF и CSRT, рассчитаны плотный и разреженный оптические потоки, получены маски переднего плана. В контролируемой задаче коррекции дисторсии PSNR достигает 31,03 дБ. Сравнение Bicubic и Real-ESRGAN показывает, что выбранная интерполяция точнее воспроизводит эталон по PSNR и SSIM; OCR дополнительно измеряет сохранность конкретной текстовой строки.

Для галереи из 80 фрагментов реализованы поиск CLIP по тексту, поиск ResNet-18 по изображению и обучение 64-мерной triplet-проекции. Проверка на отложенных данных показывает различие между минимизацией обучающих потерь и качеством поиска. Универсальные zero-shot-модели дают различные рамки и маски в зависимости от запроса; это учитывается при подготовке разметки.

Поиск в RuTube представлен локальным индексом из 81 кадра и сохранённой выдачей Streamlit с моментами 144, 2 и 148 секунд. Для разметки подготовлены два COCO-архива и XML со скелетами. В MLflow записаны восемь запусков деревьев и пять MLP, зарегистрированы две версии дерева и проверена повторная загрузка. Для YOLOv8s-pose и YOLO11s-pose сохранены лучшие веса, истории 20 эпох и изображения тестовых предсказаний.

Практический результат работы — согласованный набор исходных данных, программ, численных наблюдений, изображений и моделей. В каждом опыте различаются задача алгоритма, формат его выхода и критерий

оценки. Это позволяет сопоставлять методы на одном материале и расширять набор изображений с сохранением воспроизводимости вычислений.

Список использованных источников

1. Методические указания к лабораторной работе № 4 «Прикладные методы компьютерного зрения». КНИТУ-КАИ, Казань, 2026. Файл Metodichka_LR4_Andreev.docx.

2. Основное изображение Treponema pallidum. URL: https://pcr.news/upload/iblock/3a5/3a5ebdd65880badcc702e1bed5c07618.jpg

3. Ultralytics: Pose, Track, YOLO-World, FastSAM. URL: https://docs.ultralytics.com/

4. MediaPipe Pose Landmarker. URL: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

5. OpenCV: Tracking, Camera Calibration, Optical Flow. URL: https://docs.opencv.org/4.x/

6. Wang X. et al. ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. ECCV Workshops, 2018.

7. Real-ESRGAN, веса x4. URL: https://huggingface.co/ai-forever/Real-ESRGAN

8. Radford A. et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.

9. Schroff F. et al. FaceNet: A Unified Embedding for Face Recognition and Clustering. CVPR, 2015.

10. Kirillov A. et al. Segment Anything. ICCV, 2023.

11. CVAT: форматы аннотаций и работа с задачами. URL: https://docs.cvat.ai/docs/

12. MLflow: tracking и model registry. URL: https://mlflow.org/docs/latest/

13. Streamlit documentation. URL: https://docs.streamlit.io/

14. yt-dlp. URL: https://github.com/yt-dlp/yt-dlp

15. Наглядная микробиология #5 Бактерии (Bacteria). URL: https://rutube.ru/video/0b337cb693454b8b734f4d4b87f856a8/

16. TreponemaPallidum.jpg. Photo Credit: Content Providers(s): CDC/ Dr. David Cox. Лицензия: Public domain. URL: https://commons.wikimedia.org/wiki/File:TreponemaPallidum.jpg

17. Treponema pallidum.jpg. Photo credit: Content providers(s): CDC / Dr. David Cox. Лицензия: Public domain. URL: https://commons.wikimedia.org/wiki/File:Treponema_pallidum.jpg

18. Treponema pallidum 01.png. CDC. Лицензия: Public domain. URL: https://commons.wikimedia.org/wiki/File:Treponema_pallidum_01.png

19. Treponemapallidum.jpg. NIAID. Лицензия: Public domain. URL: https://commons.wikimedia.org/wiki/File:Treponemapallidum.jpg

20. Treponema pallidum Bacteria (Syphilis) - NIAID.jpg. NIAID. Лицензия: CC BY 2.0. URL: https://commons.wikimedia.org/wiki/File:Treponema_pallidum_Bacteria_(Syphilis)_-_NIAID.jpg

21. Syphilis Bacteria (16842837731).jpg. NIAID. Лицензия: CC BY 2.0. URL: https://commons.wikimedia.org/wiki/File:Syphilis_Bacteria_(16842837731).jpg

22. Ultramicroscope time-lapse of syphilis parasite-Comandon-1910.ogv. Jean Comandon. Лицензия: Public domain. URL: https://commons.wikimedia.org/wiki/File:Ultramicroscope_time-lapse_of_syphilis_parasite-Comandon-1910.ogv